

Agenda

Hack, the big picture: simulation from gates to computer

Hack Hardware Architecture

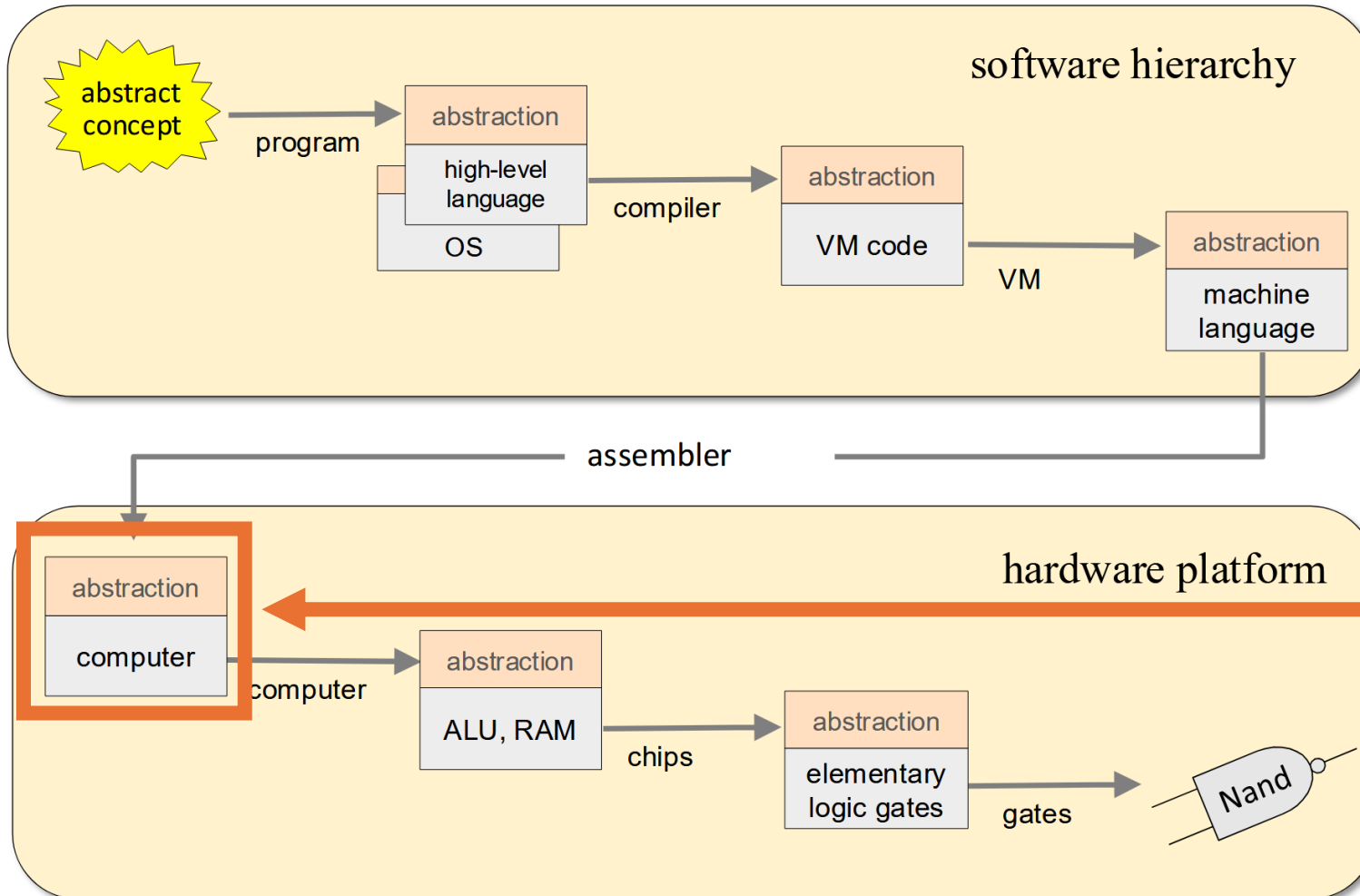
- Hack Assembly Language

- Machine Code

- CPU Design and Implementation

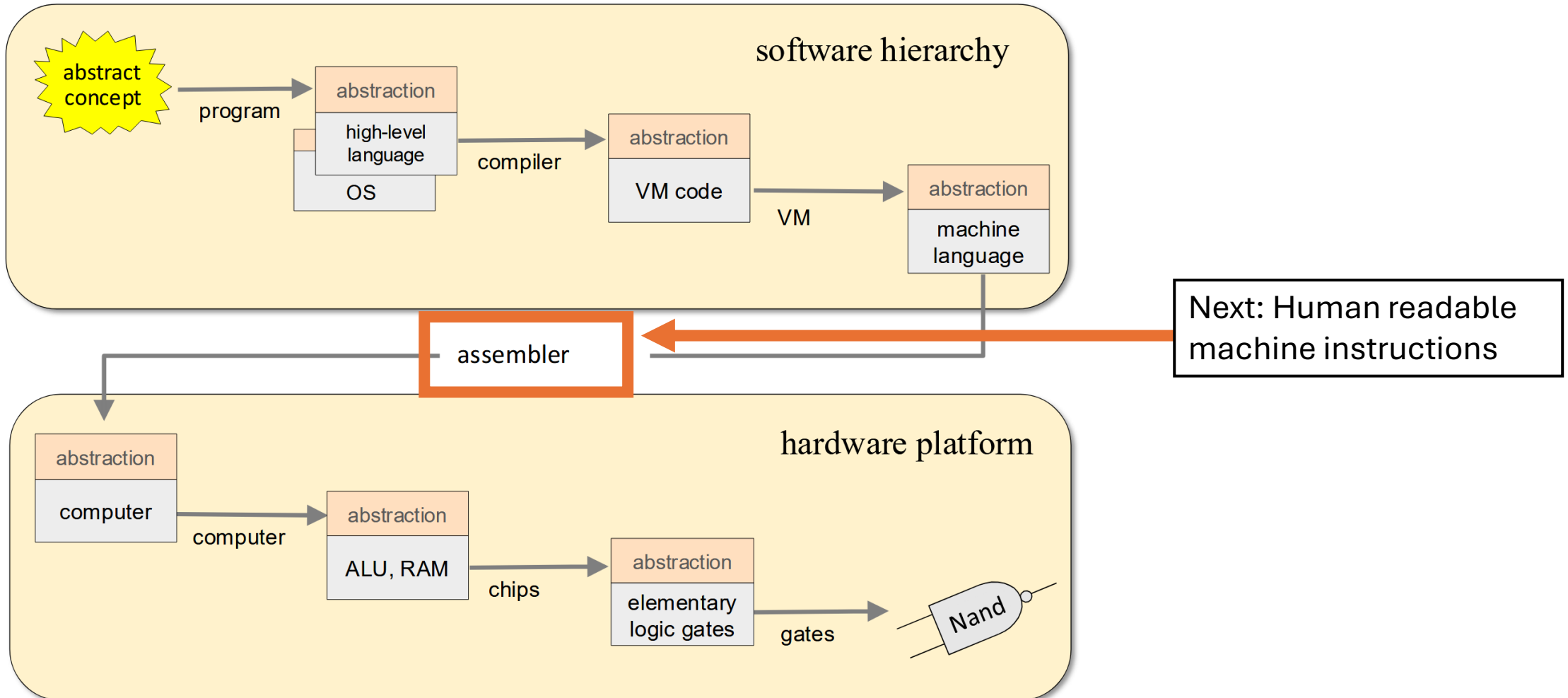
- Computer Design and Implementation

The big picture: Hack



Next: How our hardware components fit together and how we can program it

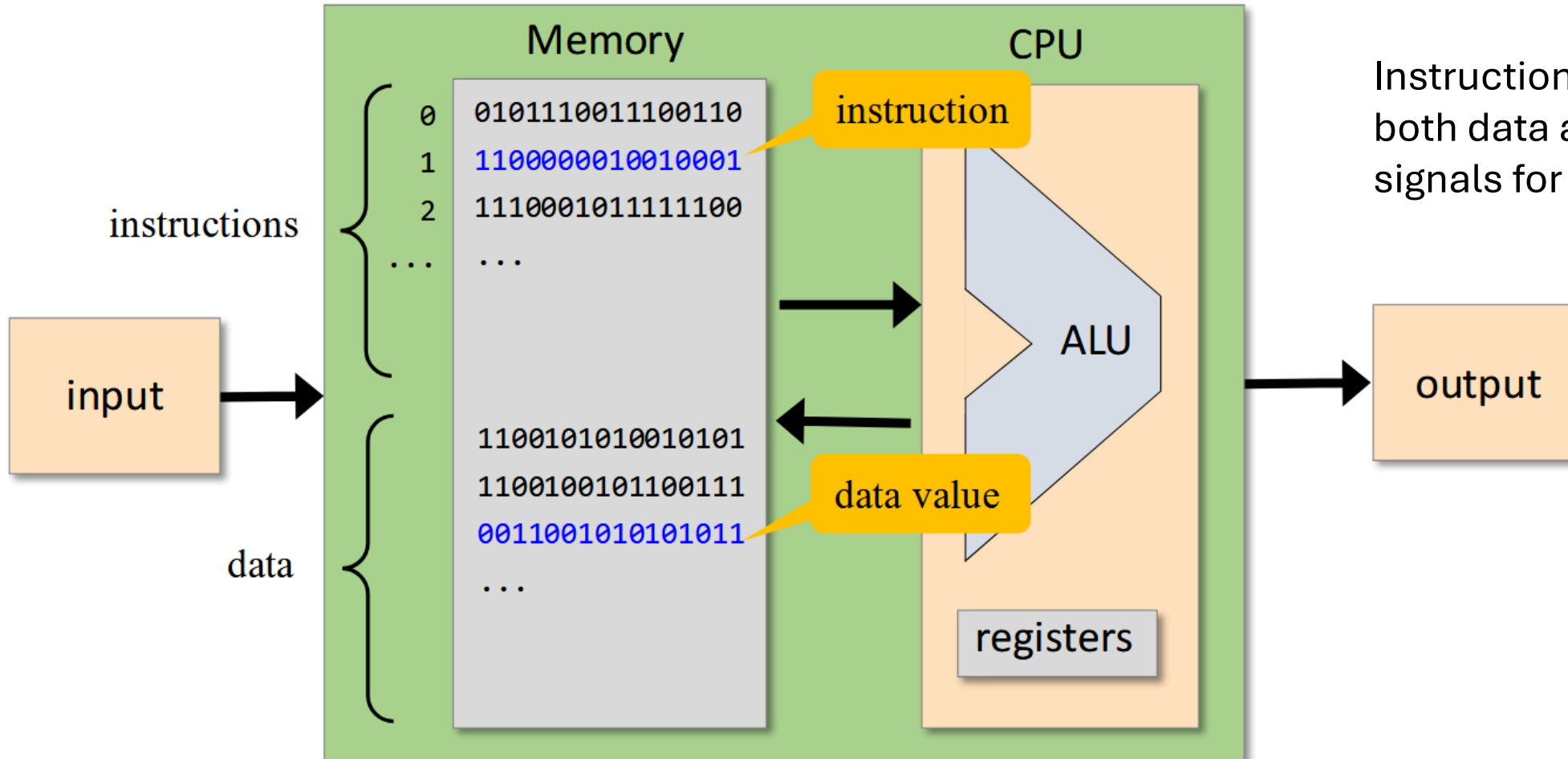
The big picture: Hack



Machine Code

Recall: The stored program model of the von Neuman architecture

Instructions can contain both data and control signals for the ALU



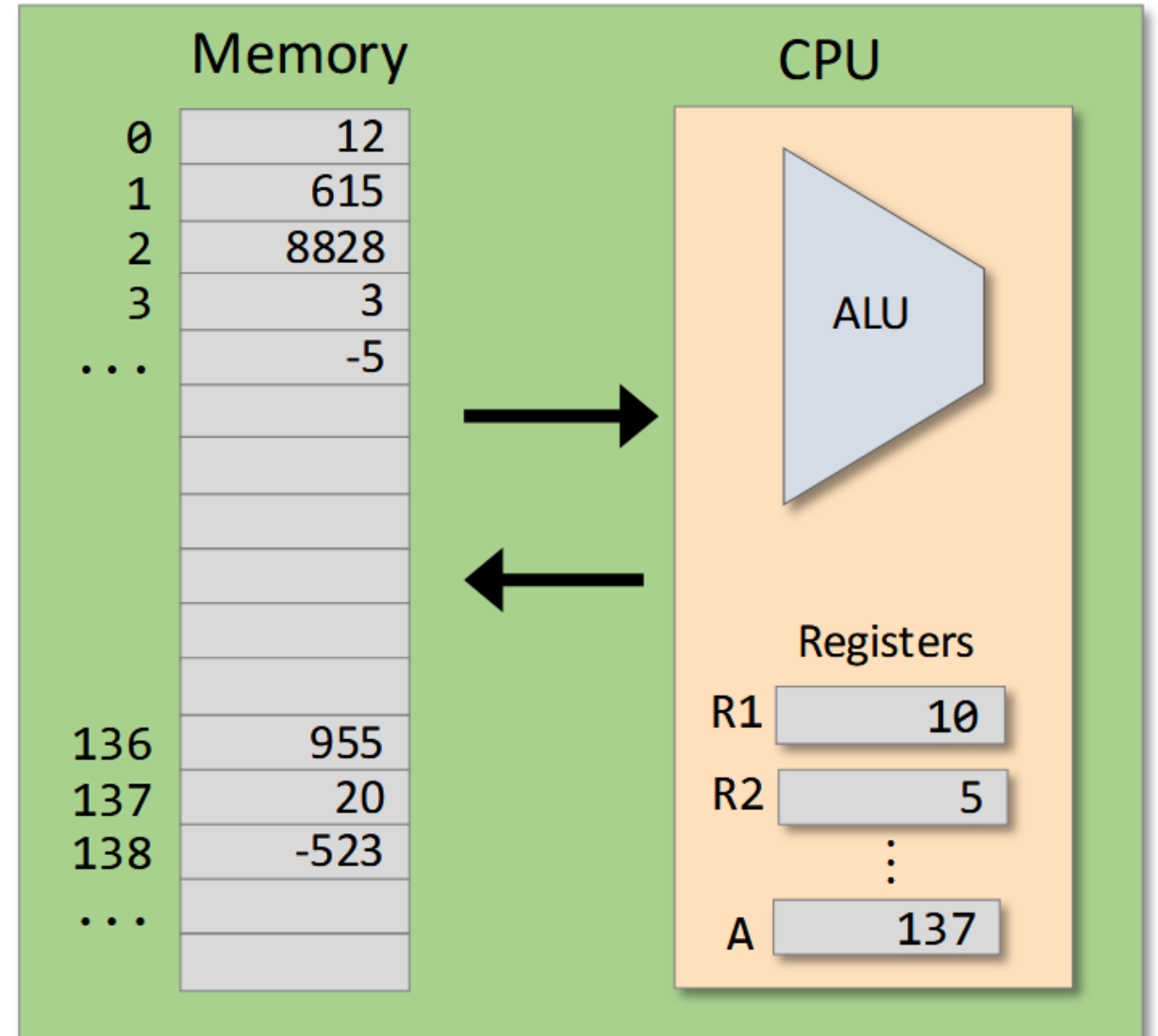
Registers

Data registers (A,D):
Hold data values

Address register (PC):
Holds an address

Instruction register (IR):
Holds an instruction

RAM (R1, R2, etc):
Holds data or instructions



From symbolic language to machine language

Symbolic program

```
// Sets R2 to abs(R1)
// R2 ← R1
mov R2,R1
// if (R2 > 0) goto cont
jgt R2,CONT

// R2 ← -R1
movi R2,0
sub R2,R1

CONT:
// Here R2 = abs(R1)
...
```

Assembly
(language)

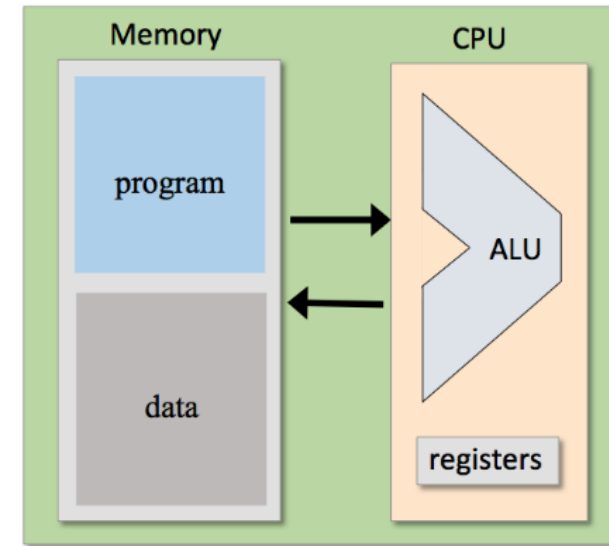
translate

Binary code

```
0101111100111100
1010101010101010
1100000010101010
1011000010000001
...
```

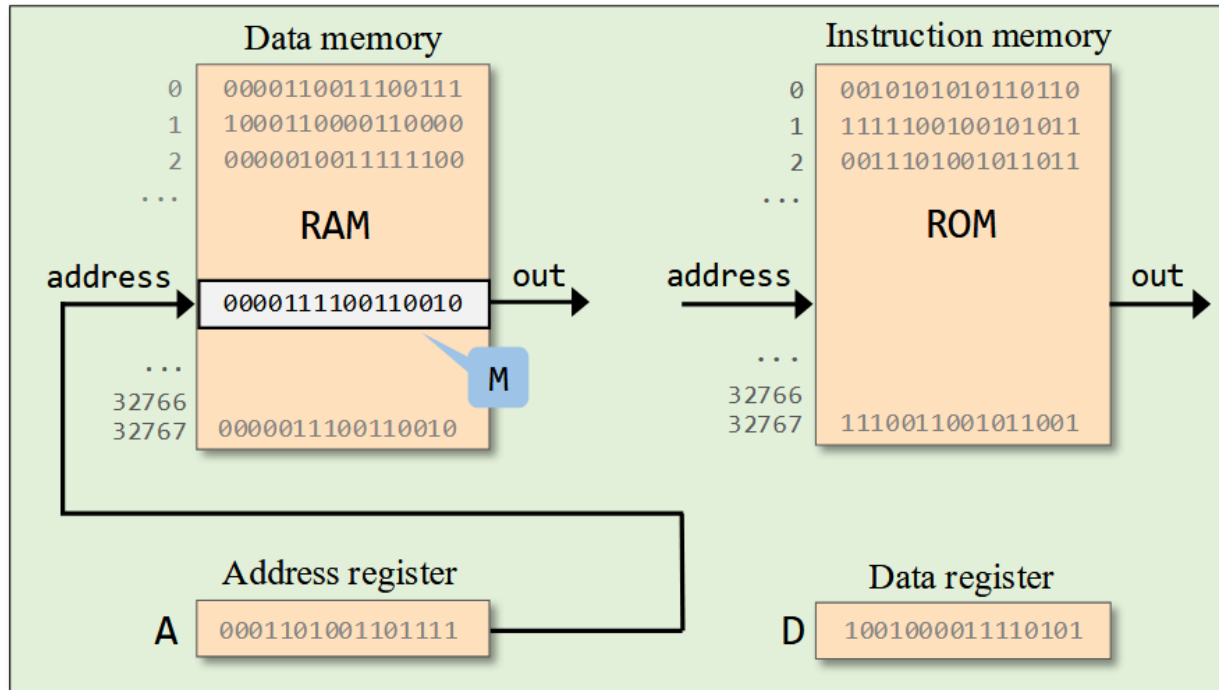
Assembler
(tool)

load and
execute



Assembly language needs to be compiled to binary code (machine language). Machine language is tied to a particular hardware.

Hack separates data from instructions



RAM

- Read-write data memory
- Addressed by the A register
- The selected memory location, RAM[A], is referred to as M

ROM

The program is stored in Read Only memory (ROM)

Hack assembly basics

Three storage types:

- D register (data values)
- A register (address value)
- M refers to a location $M[A]$ in RAM

Two instruction types:

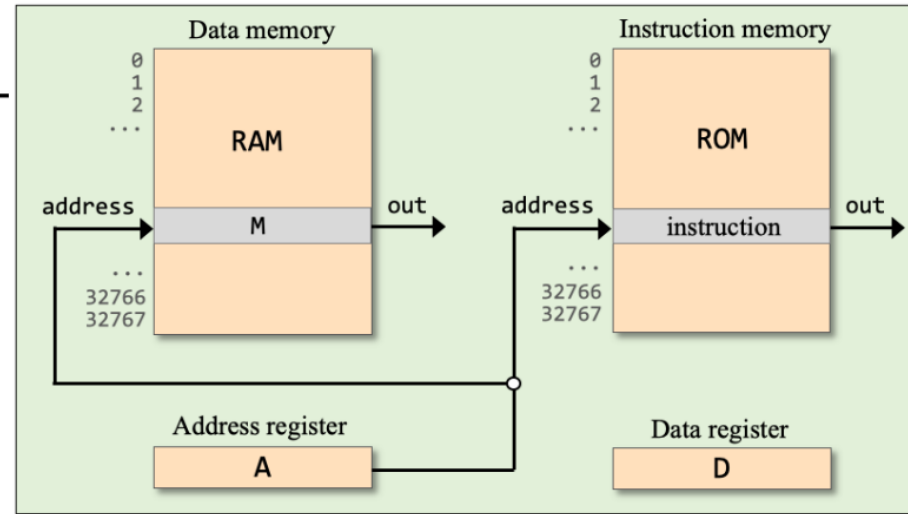
- A-instruction (Load a value into the A register)
- C-instruction (Compute)

A-instruction

Hack instructions

Instruction set

- ➔ A - instruction (*address*)
- C - instruction (*compute*)



Syntax:

@xxx

where xxx is a
non-negative integer

Example

@19

Semantics

$A \leftarrow 19$

Side effects:

- RAM[A] (denoted M) becomes selected
- ROM[A] becomes selected

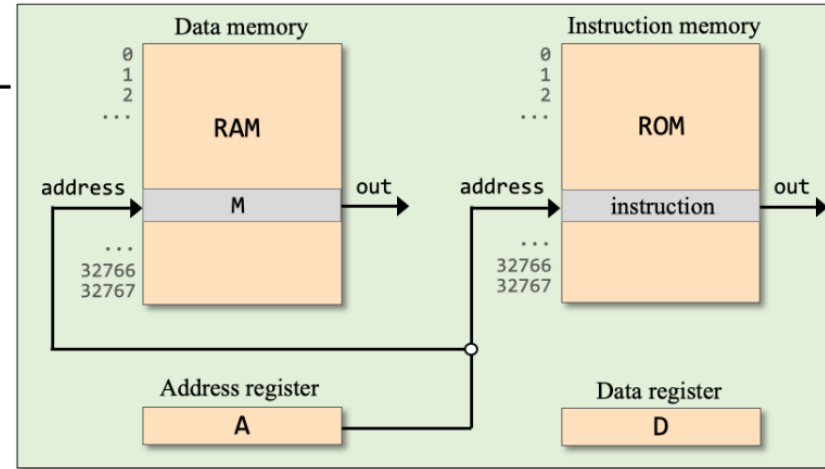
C-instruction

Hack instructions

Instruction set

- A - instruction (*address*)

➔ C - instruction (*compute*)



Syntax:

$reg = \{0 | 1 | -1\}$

where $reg = \{A | D | M\}$

$reg_1 = reg_2$

where $reg_1 = \{A | D | M\}$
 $reg_2 = [-] \{A | D | M\}$

$reg = reg_1 \text{ op } reg_2$

where $reg, reg_1 = \{A | D | M\}$
 $reg_2 = \{A | D | M | 1\}$
 $op = \{+ | - | \& | !\}$

Examples:

D=0
A=-1
M=1
...

D=A
D=M
M=-M
...

D=D+M
A=A-1
M=D+1
...

(Complete / formal
syntax, later).

Exercise: Convert to Hack assembly

// $D \leftarrow 2$

// $D \leftarrow 1954$

// $D = D + 23$

// $M[4] \leftarrow D$

// $M[6] \leftarrow M[4] - 10$

Exercise: Write Hack assembly

- a) $M[100] \leftarrow 0$
- b) $M[100] \leftarrow 17$
- c) $RAM[100] \leftarrow RAM[200]$
- d) $RAM[3] \leftarrow RAM[3] - 15$
- e) $RAM[3] \leftarrow RAM[4] + 1$

Syntax:

$reg = \{0|1|-1\}$

where $reg = \{A|D|M\}$

$reg_1 = reg_2$

where $reg_1 = \{A|D|M\}$

$reg_2 = [-] \{A|D|M\}$

$reg = reg_1 \text{ op } reg_2$

where $reg, reg_1 = \{A|D|M\}$

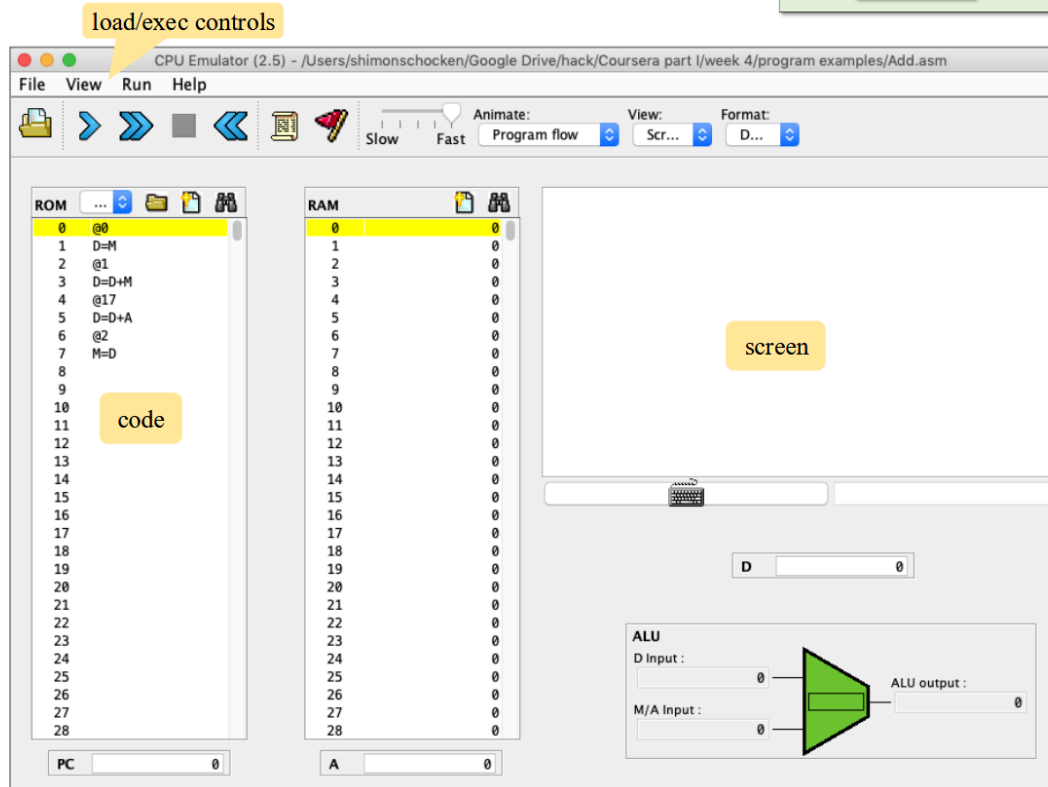
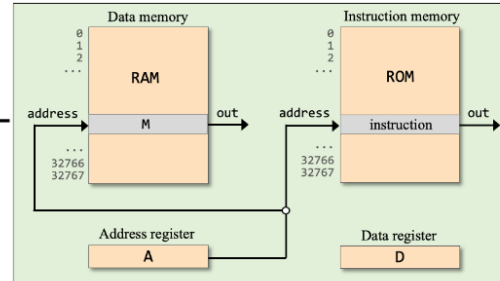
$reg_2 = \{A|D|M|1\}$

$op = \{+|-|\&|!\}$

Aside: Hack CPU Emulator

The CPU emulator

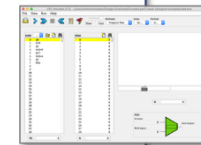
- Software that emulates the Hack CPU
- Part of the Nand to Tetris IDE



Add.asm (example)

```
// Computes: RAM[2] = RAM[0] + RAM[1] + 17
// D = RAM[0]
@0
D=M
// D = D + RAM[1]
@1
D=D+M
// D = D + 17
@17
D=D+A
// RAM[2] = D
@2
M=D
```

Load into the
CPU emulator



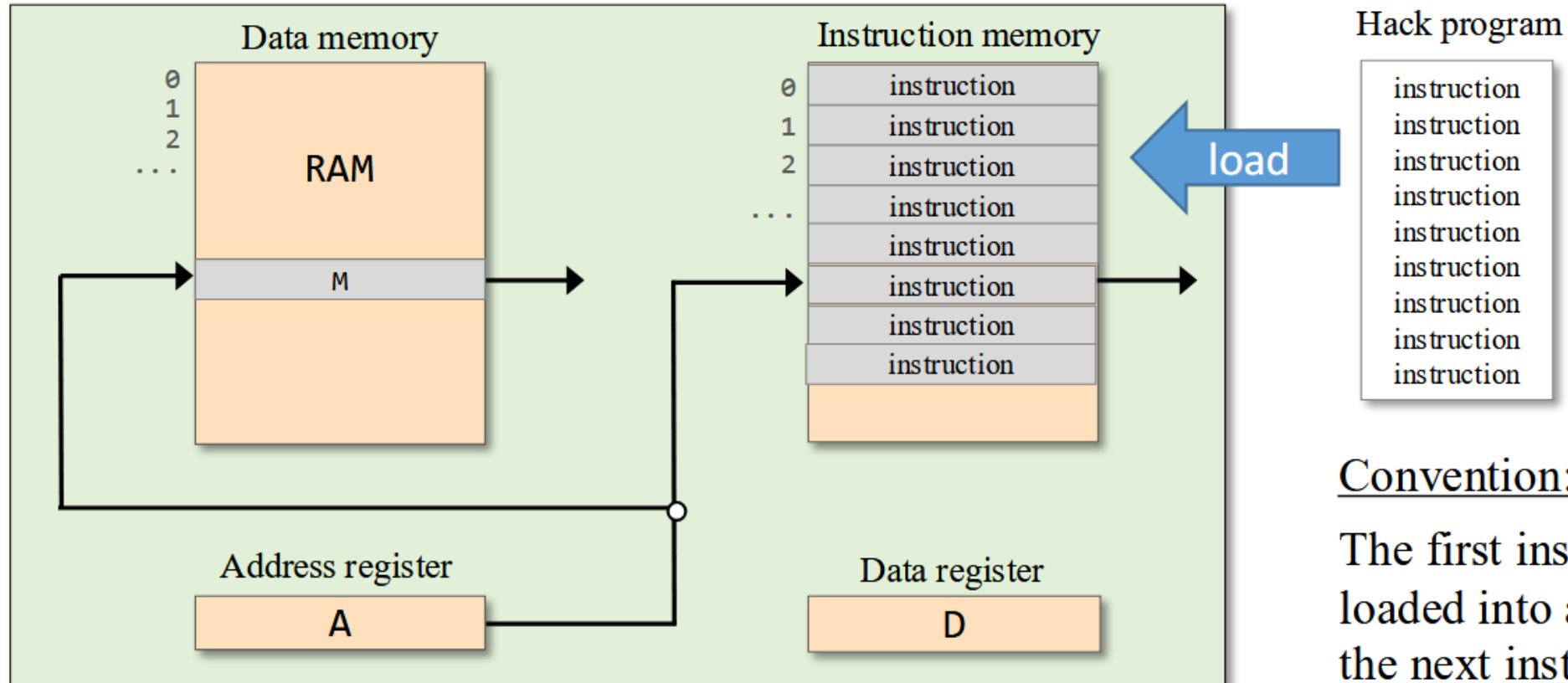
Binary

```
0000000000000000
1000010010001101
0000000000000001
1010011001100001
0000000000010001
1001111100110011
0000000000000010
1110010010010011
```

Execute in the
CPU emulator

When loading a symbolic program into our CPU emulator, the emulator translates it into binary code (using a built-in assembler).

Loading and running a program



Convention:

The first instruction is loaded into address 0, the next instruction into address 1, and so on.

To execute the program, process each instruction sequentially

Loading and running a program in Hack

Loading a program places each program instruction in memory

The PC is initialized to start at 0

Until a branches, each instruction is executed sequentially

E.g. Each step, we fetch the instruction based on the PC, execute it, and then increment the PC.

A branch tests a condition. If true, we change the value of the PC

Branching

What about conditionals and loops?

We want to test a condition and possibly jump to a different instruction

Hack syntax

- Use an A-instruction to select an address
- Use a C-instruction to jump to that address

Example:

```
0;JMP // Jump to execute the instruction stored in ROM[A]
```


Branching

```
// Example  
// if (D > 0) goto 6  
@6  
D;JGT
```

Branch command syntax:

```
D;JGT // if D > 0 jump  
D;JGE // if D ≥ 0 jump  
D;JLT // if D < 0 jump  
D;JLE // if D ≤ 0 jump  
D;JEQ // if D = 0 jump  
D;JNE // if D ≠ 0 jump  
0;JMP // unconditional jump
```

In each case,
jumps to the
instruction
stored in
ROM[A]

Example: Branching

// Write asm for the following
// goto 6

	ROM
0	
1	
2	goto 6
3	
4	
5	
6	
7	
8	
9	
10	
11	

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC
2

A

D

Example: Branching

```
// Write asm for the following  
// if (RAM[3] < 100) goto 9
```

	ROM
0	
1	
2	if(..)goto
3	
4	
5	
6	
7	
8	
9	
10	
11	

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC
2

A

D

Example: Branching

```
// Write asm for the following  
// if (RAM[8] != -5) goto 9
```

	ROM
0	
1	
2	if(..)goto
3	
4	
5	
6	
7	
8	
9	
10	
11	

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC
2

A

D

Variables in hack assembly

Pseudocode (example)

```
...  
i = 1  
sum = 0  
...  
sum = sum + i  
i = i + 1  
...
```

write



Hack assembly

```
...  
// i = 1  
@i  
M=1  
...  
// sum = 0  
@sum  
M=0  
...  
// sum = sum + i  
@i  
D=M  
@sum  
M=D+M  
...  
// i = i + 1  
@i  
M=M+1  
...
```

Symbolic programming

- The code writer is allowed to create and use symbolic variables, as needed
- We assume that there is an agent who knows how to bind these symbols to sensible RAM addresses

This agent is the *assembler*

For example

- If the assembler will bind `i` and `sum` to, say, 16 and 17, every instruction `@i` and `@sum` will end up selecting `RAM[16]` and `RAM[17]`
- Invisible to the code writer
- The result: a powerful, low-level, *variables abstraction*.

Idea: The programmer does not need to assign memory addresses explicitly

Exercise: Hack variables

Typical instructions:

`@constant` $A \leftarrow \text{constant}$

`@symbol` $A \leftarrow \text{the constant which is bound to } \textit{symbol}$

D=0
M=1
D=-1
M=0
...

D=M
A=M
M=D
D=A
...

D=D+A
D=A+1
D=D+M
M=M-1
...

Use only the above instructions

// sum = 0

?

// x = 512

?

// n = n - 1

?

// sum = sum + x

?

Virtual register names

Pre-defined symbols in the Hack language

<u>symbol</u>	<u>value</u>
R0	0
R1	1
R2	2
...	...
R15	15

RAM	
0	
1	
2	
...	
15	
16	
17	
...	
32767	

16 “built-in variables” named R0...R15
Sometimes referred to as “virtual registers”

Example:

```
// Sets R1 to 2 * R0  
// Usage: Enter a value in R0
```

Labels

Idea: The programmer does not need to know the ROM addresses of instructions

Example (pseudocode)

```
i = 1000
LOOP:
  if (i = 0) goto CONT
  i = i - 1
  goto LOOP
CONT:
  ...
```

write



Hack assembly

```
// i = 1000
@1000
D=A
@i
M=D
(LABEL)
// if(i = 0) goto CONT
@i
D=M
@CONT
D;JEQ
// i = i - 1
@i
M=M-1
// goto LOOP
@LABEL
0;JMP
(CONT)
...
```

Label declaration in the Hack assembly language:

(*sym*)

Results in binding *sym* to the address of the next instruction

In this example:

LOOP is bound to 4

CONT is bound to 12

(done by the assembler;
The code writer doesn't care
about these numbers)

Example: Variables and Labels

Convert to assembly

```
i = 1000  
LOOP:  
  if (i == 0) goto CONT  
  i = i - 1  
  goto LOOP  
CONT:  
  ...
```

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	9876
5	
6	
7	
8	
9	
10	
11	

PC

0

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Example: variables and labels

```
@1000
D=A
@i
M=D
(LOOP)
@i
D=M
@CONT
D;JEQ
@i
M=M-1
@LOOP
0;JMP
(CONT)
```

Assume: $i \rightarrow 4$
LOOP $\rightarrow 4$
CONT $\rightarrow 12$

	ROM
0	@1000
1	D=A
2	@4
3	M=D
4	@4
5	D=M
6	@12
7	D;JEQ
8	@4
9	M=M-1
10	@4
11	0;JMP
12	...

	Memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

PC

A

D

Marking the end of a program

Hack convention is to end the program on an infinite loop

```
(END)  
  @END  
  0;JMP
```

Assembly Language Summary

- Two assembly commands: A-instructions and C-instructions
- Symbols can be any sequence of letters, digits, underscore, dot, dollar sign, and colon. Symbols cannot begin with a digit
- Constants must be non-negative and written in decimal notation
- Comments begin with //
- White space is ignored
- Case conventions: Assembly mnemonics (KBD, SCREEN, R0, etc) must be all uppercase. Symbols and labels are case sensitive
- Assembly programs have the .asm extension

Machine Code

Assembly code maps to the operations that our computer can execute

However, we need to convert these instructions to binary so they can be converted to

- Control signals for the ALU
- Values for the A register
- Control signals for setting the PC based on the result of the ALU
- Read/write signals for memory

.hack files store machine code

Example: @12 → 0000 0000 0000 1100

Hack Instructions to Machine Code: A

<u>A instruction</u>	Symbolic: $@xxx$	(xxx is a decimal value ranging from 0 to 32767, or a symbol bound to such a decimal value)
	Binary: $0\ vvvvvvvvvvvvvvvvv$	($vv \dots v = 15\text{-bit value of } xxx$)

Exercise: Convert the following commands to binary

@9

@128

Hack Instructions to Machine Code: C

C instruction

Symbolic: *dest = comp; jump*

(*comp* is mandatory.

If *dest* is empty, the = is omitted;

If *jump* is empty, the ; is omitted)

Binary: **111***acccccddjjj*

<i>comp</i>		<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>dest</i>	<i>d</i>	<i>d</i>	<i>d</i>	Effect: store <i>comp</i> in:
0		1	0	1	0	1	0	null	0	0	0	the value is not stored
1		1	1	1	1	1	1	M	0	0	1	RAM[A]
-1		1	1	1	0	1	0	D	0	1	0	D register (reg)
D		0	0	1	1	0	0	DM	0	1	1	RAM[A] and D reg
A	M	1	1	0	0	0	0	A	1	0	0	A reg
!D		0	0	1	1	0	1	AM	1	0	1	A reg and RAM[A]
!A	!M	1	1	0	0	0	1	AD	1	1	0	A reg and D reg
-D		0	0	1	1	1	1	ADM	1	1	1	A reg, D reg, and RAM[A]
-A	-M	1	1	0	0	1	1					
D+1		0	1	1	1	1	1	<i>jump</i>	<i>j</i>	<i>j</i>	<i>j</i>	Effect:
A+1	M+1	1	1	0	1	1	1	null	0	0	0	no jump
D-1		0	0	1	1	1	0	JGT	0	0	1	if <i>comp</i> > 0 jump
A-1	M-1	1	1	0	0	1	0	JEQ	0	1	0	if <i>comp</i> = 0 jump
D+A	D+M	0	0	0	0	1	0	JGE	0	1	1	if <i>comp</i> ≥ 0 jump
D-A	D-M	0	1	0	0	1	1	JLT	1	0	0	if <i>comp</i> < 0 jump
A-D	M-D	0	0	0	1	1	1	JNE	1	0	1	if <i>comp</i> ≠ 0 jump
D&A	D&M	0	0	0	0	0	0	JLE	1	1	0	if <i>comp</i> ≤ 0 jump
D A	D M	0	1	0	1	0	1	JMP	1	1	1	unconditional jump

a == 0 *a* == 1

Exercise: C instruction

Symbolic: *dest = comp; jump* (*comp* is mandatory.
If *dest* is empty, the = is omitted;
If *jump* is empty, the ; is omitted)

Binary: 111*acccccddjjj*

Convert the following
command to binary
machine code: M=M+1

<i>comp</i>		<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>dest</i>	<i>d</i>	<i>d</i>	<i>d</i>	Effect: store <i>comp</i> in:
0		1	0	1	0	1	0	null	0	0	0	the value is not stored
1		1	1	1	1	1	1	M	0	0	1	RAM[A]
-1		1	1	1	0	1	0	D	0	1	0	D register (reg)
D		0	0	1	1	0	0	DM	0	1	1	RAM[A] and D reg
A	M	1	1	0	0	0	0	A	1	0	0	A reg
!D		0	0	1	1	0	1	AM	1	0	1	A reg and RAM[A]
!A	!M	1	1	0	0	0	1	AD	1	1	0	A reg and D reg
-D		0	0	1	1	1	1	ADM	1	1	1	A reg, D reg, and RAM[A]
-A	-M	1	1	0	0	1	1					
D+1		0	1	1	1	1	1					
A+1	M+1	1	1	0	1	1	1					
D-1		0	0	1	1	1	0					
A-1	M-1	1	1	0	0	1	0					
D+A	D+M	0	0	0	0	1	0					
D-A	D-M	0	1	0	0	1	1					
A-D	M-D	0	0	0	1	1	1					
D&A	D&M	0	0	0	0	0	0					
D A	D M	0	1	0	1	0	1					

<i>jump</i>	<i>j</i>	<i>j</i>	<i>j</i>	Effect:
null	0	0	0	no jump
JGT	0	0	1	if <i>comp</i> > 0 jump
JEQ	0	1	0	if <i>comp</i> = 0 jump
JGE	0	1	1	if <i>comp</i> ≥ 0 jump
JLT	1	0	0	if <i>comp</i> < 0 jump
JNE	1	0	1	if <i>comp</i> ≠ 0 jump
JLE	1	1	0	if <i>comp</i> ≤ 0 jump
JMP	1	1	1	unconditional jump

a == 0 *a* == 1

Exercise: C instruction

Symbolic: *dest = comp; jump* (*comp* is mandatory.
If *dest* is empty, the = is omitted;
If *jump* is empty, the ; is omitted)

Binary: 111*acccccddjjj*

Convert the following
command to binary
machine code: D=0

<i>comp</i>		<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>dest</i>	<i>d</i>	<i>d</i>	<i>d</i>	Effect: store <i>comp</i> in:
0		1	0	1	0	1	0	null	0	0	0	the value is not stored
1		1	1	1	1	1	1	M	0	0	1	RAM[A]
-1		1	1	1	0	1	0	D	0	1	0	D register (reg)
D		0	0	1	1	0	0	DM	0	1	1	RAM[A] and D reg
A	M	1	1	0	0	0	0	A	1	0	0	A reg
!D		0	0	1	1	0	1	AM	1	0	1	A reg and RAM[A]
!A	!M	1	1	0	0	0	1	AD	1	1	0	A reg and D reg
-D		0	0	1	1	1	1	ADM	1	1	1	A reg, D reg, and RAM[A]
-A	-M	1	1	0	0	1	1					
D+1		0	1	1	1	1	1					
A+1	M+1	1	1	0	1	1	1					
D-1		0	0	1	1	1	0					
A-1	M-1	1	1	0	0	1	0					
D+A	D+M	0	0	0	0	1	0					
D-A	D-M	0	1	0	0	1	1					
A-D	M-D	0	0	0	1	1	1					
D&A	D&M	0	0	0	0	0	0					
D A	D M	0	1	0	1	0	1					

<i>jump</i>	<i>j</i>	<i>j</i>	<i>j</i>	Effect:
null	0	0	0	no jump
JGT	0	0	1	if <i>comp</i> > 0 jump
JEQ	0	1	0	if <i>comp</i> = 0 jump
JGE	0	1	1	if <i>comp</i> ≥ 0 jump
JLT	1	0	0	if <i>comp</i> < 0 jump
JNE	1	0	1	if <i>comp</i> ≠ 0 jump
JLE	1	1	0	if <i>comp</i> ≤ 0 jump
JMP	1	1	1	unconditional jump

a == 0 *a* == 1

Exercise: C instruction

Symbolic: $dest = comp; jump$ (*comp* is mandatory.
If *dest* is empty, the = is omitted;
If *jump* is empty, the ; is omitted)

Binary: $111acccccddjjj$

Convert the following
command to binary
machine code: $D=D+A$

<i>comp</i>		<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>dest</i>	<i>d</i>	<i>d</i>	<i>d</i>	Effect: store <i>comp</i> in:
0		1	0	1	0	1	0	null	0	0	0	the value is not stored
1		1	1	1	1	1	1	M	0	0	1	RAM[A]
-1		1	1	1	0	1	0	D	0	1	0	D register (reg)
D		0	0	1	1	0	0	DM	0	1	1	RAM[A] and D reg
A	M	1	1	0	0	0	0	A	1	0	0	A reg
!D		0	0	1	1	0	1	AM	1	0	1	A reg and RAM[A]
!A	!M	1	1	0	0	0	1	AD	1	1	0	A reg and D reg
-D		0	0	1	1	1	1	ADM	1	1	1	A reg, D reg, and RAM[A]
-A	-M	1	1	0	0	1	1	<i>jump</i> <i>j</i> <i>j</i> <i>j</i> Effect:				
D+1		0	1	1	1	1	1	null	0	0	0	no jump
A+1	M+1	1	1	0	1	1	1	JGT	0	0	1	if <i>comp</i> > 0 jump
D-1		0	0	1	1	1	0	JEQ	0	1	0	if <i>comp</i> = 0 jump
A-1	M-1	1	1	0	0	1	0	JGE	0	1	1	if <i>comp</i> ≥ 0 jump
D+A	D+M	0	0	0	0	1	0	JLT	1	0	0	if <i>comp</i> < 0 jump
D-A	D-M	0	1	0	0	1	1	JNE	1	0	1	if <i>comp</i> ≠ 0 jump
A-D	M-D	0	0	0	1	1	1	JLE	1	1	0	if <i>comp</i> ≤ 0 jump
D&A	D&M	0	0	0	0	0	0	JMP	1	1	1	unconditional jump
D A	D M	0	1	0	1	0	1					

$a == 0$ $a == 1$

Exercise: C instruction

Symbolic: *dest = comp; jump* (*comp* is mandatory.
If *dest* is empty, the = is omitted;
If *jump* is empty, the ; is omitted)

Binary: 111*acccccddjjj*

Convert the following
command to binary
machine code: D;JEQ

<i>comp</i>		<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>dest</i>	<i>d</i>	<i>d</i>	<i>d</i>	Effect: store <i>comp</i> in:
0		1	0	1	0	1	0	null	0	0	0	the value is not stored
1		1	1	1	1	1	1	M	0	0	1	RAM[A]
-1		1	1	1	0	1	0	D	0	1	0	D register (reg)
D		0	0	1	1	0	0	DM	0	1	1	RAM[A] and D reg
A	M	1	1	0	0	0	0	A	1	0	0	A reg
!D		0	0	1	1	0	1	AM	1	0	1	A reg and RAM[A]
!A	!M	1	1	0	0	0	1	AD	1	1	0	A reg and D reg
-D		0	0	1	1	1	1	ADM	1	1	1	A reg, D reg, and RAM[A]
-A	-M	1	1	0	0	1	1	<i>jump</i> <i>j</i> <i>j</i> <i>j</i> Effect:				
D+1		0	1	1	1	1	1	null	0	0	0	no jump
A+1	M+1	1	1	0	1	1	1	JGT	0	0	1	if <i>comp</i> > 0 jump
D-1		0	0	1	1	1	0	JEQ	0	1	0	if <i>comp</i> = 0 jump
A-1	M-1	1	1	0	0	1	0	JGE	0	1	1	if <i>comp</i> ≥ 0 jump
D+A	D+M	0	0	0	0	1	0	JLT	1	0	0	if <i>comp</i> < 0 jump
D-A	D-M	0	1	0	0	1	1	JNE	1	0	1	if <i>comp</i> ≠ 0 jump
A-D	M-D	0	0	0	1	1	1	JLE	1	1	0	if <i>comp</i> ≤ 0 jump
D&A	D&M	0	0	0	0	0	0	JMP	1	1	1	unconditional jump
D A	D M	0	1	0	1	0	1					

a == 0 *a* == 1

CPU Design: Observations

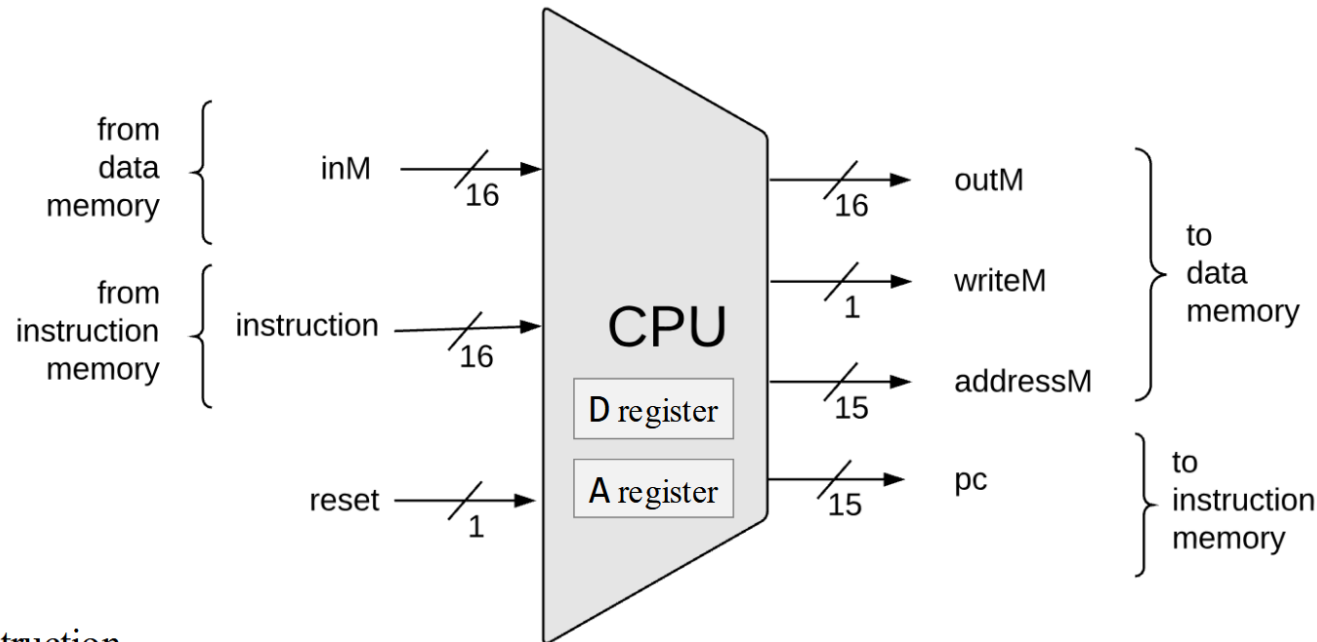
The *a* bit determines whether the source to the ALU is A or memory

The control bits of the C instruction correspond to the control bits of the ALU: *zx*, *nx*, *xy*, *ny*, *f*, *no*

The destination bits determine where the output of the ALU goes

The jump control bits can be computed from the outputs of the ALU: *zr*, *ng*

CPU: Abstraction



Instruction examples:

```
// D = RAM[5] + 1
@5
DM=M+1
...
// goto 200
@200
0; JMP
...
```

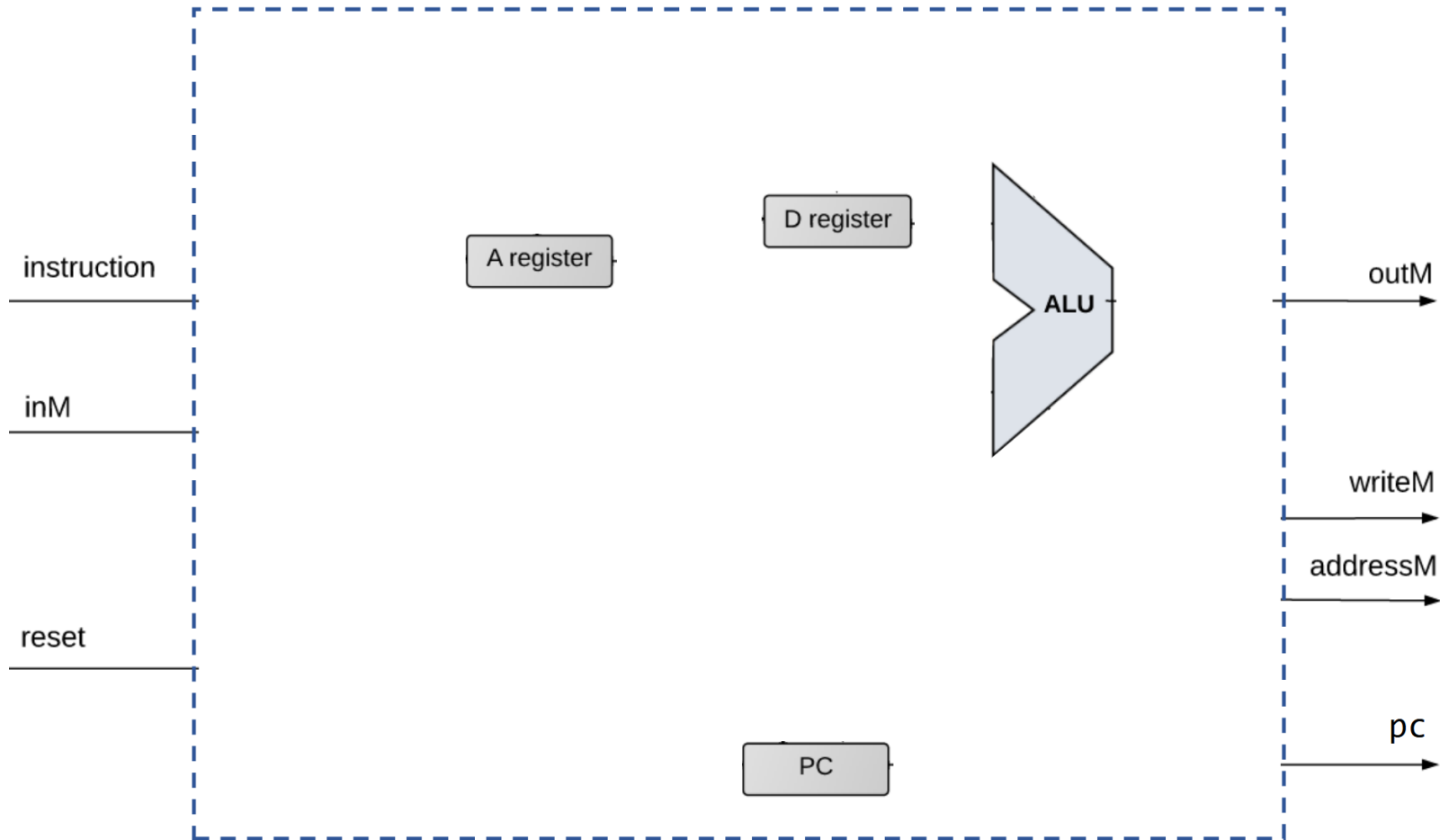
1. Executes the current instruction:

If it's an A-instruction (@xxx), sets the A register to xxx

Else (C-instruction):

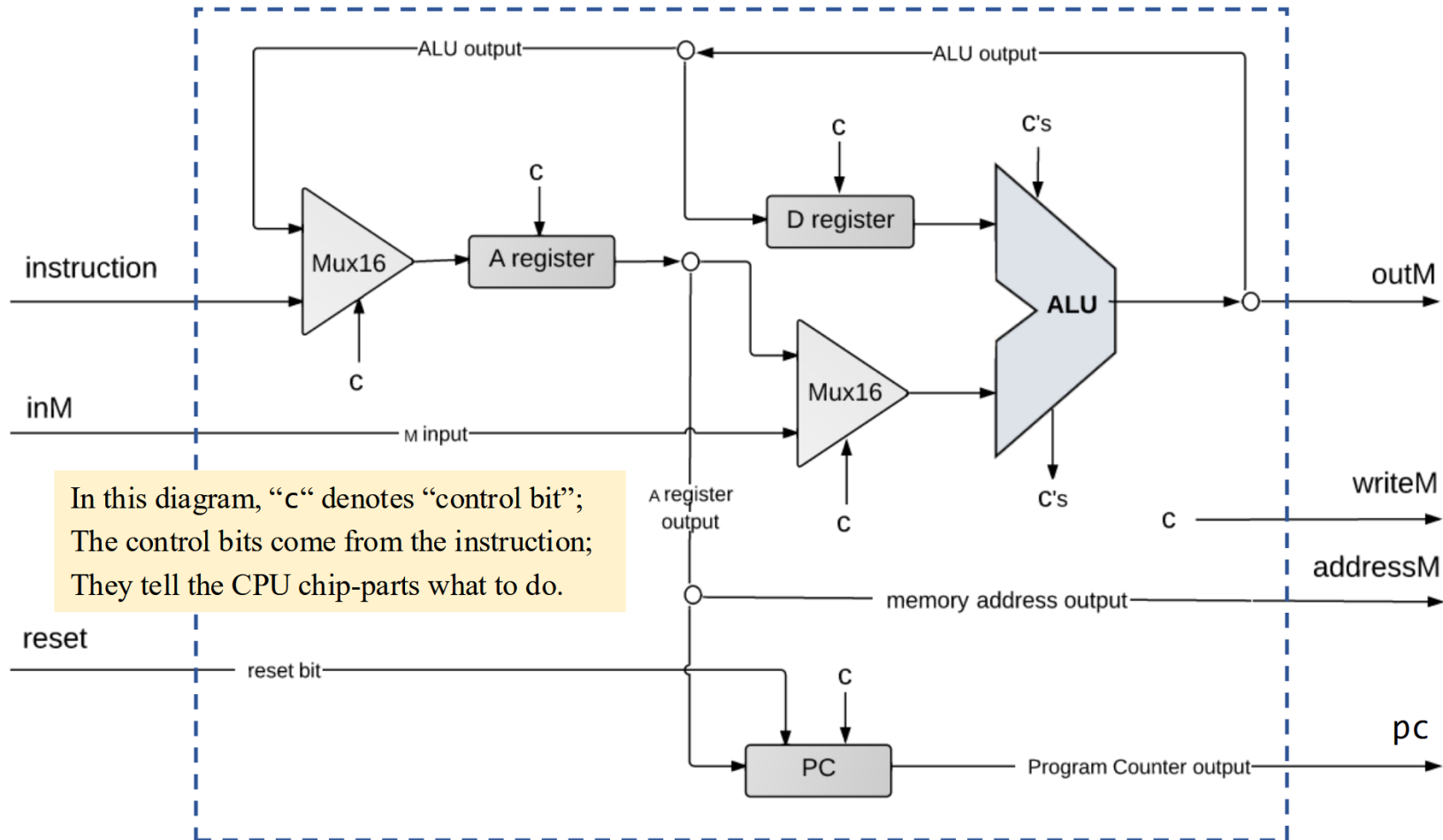
- Computes the ALU function specified by the instruction (on the values of A, D, inM)
- Puts the ALU output in A, D, outM, as specified by the instruction
- If the instruction writes to M, sets addressM to A and asserts writeM

CPU: Implementation



Hack CPU architecture

CPU: Implementation

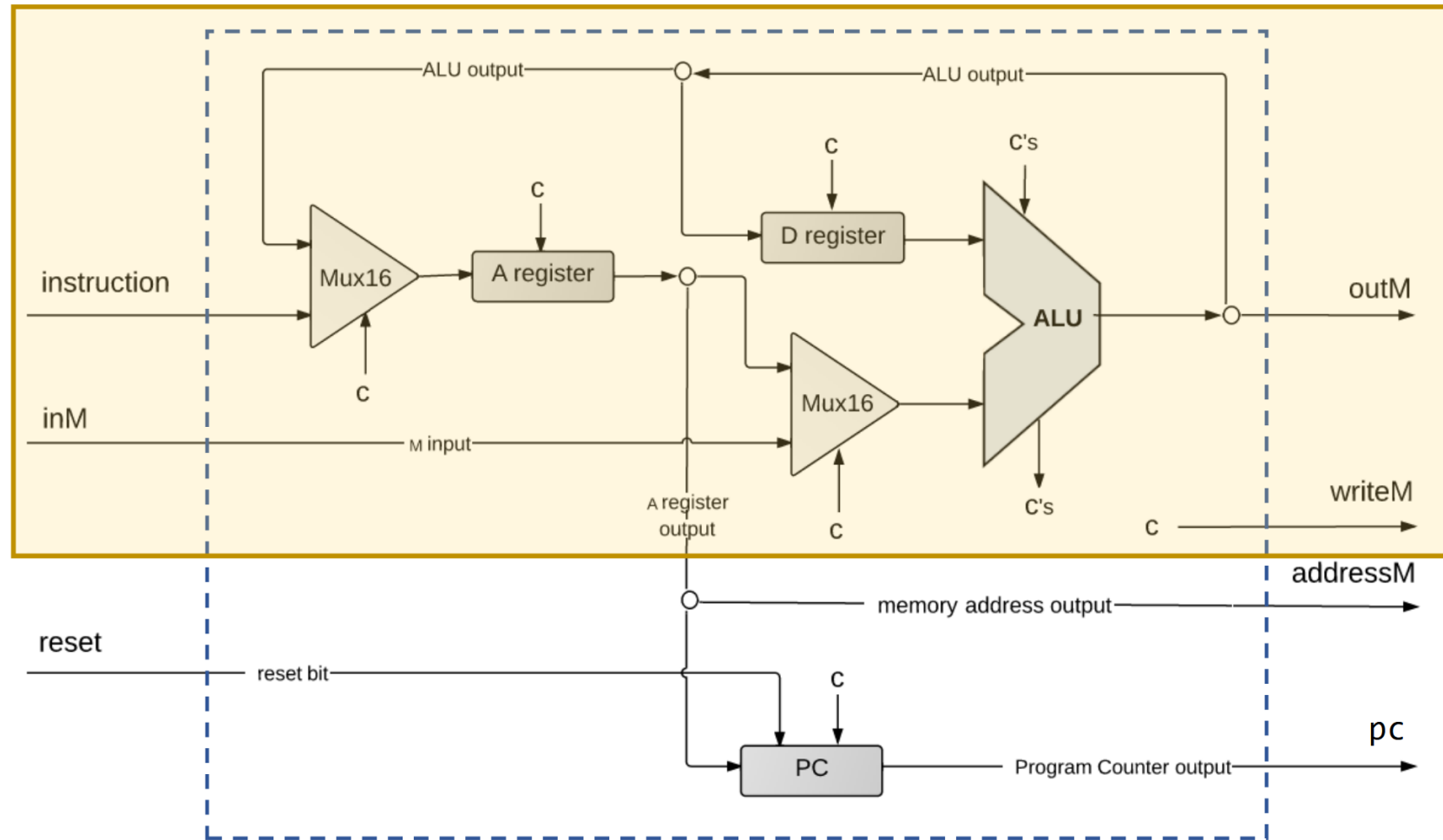


Hack CPU architecture

Example instructions:

@13
D=0
M=M+1
0;JMP

CPU: Instruction handling



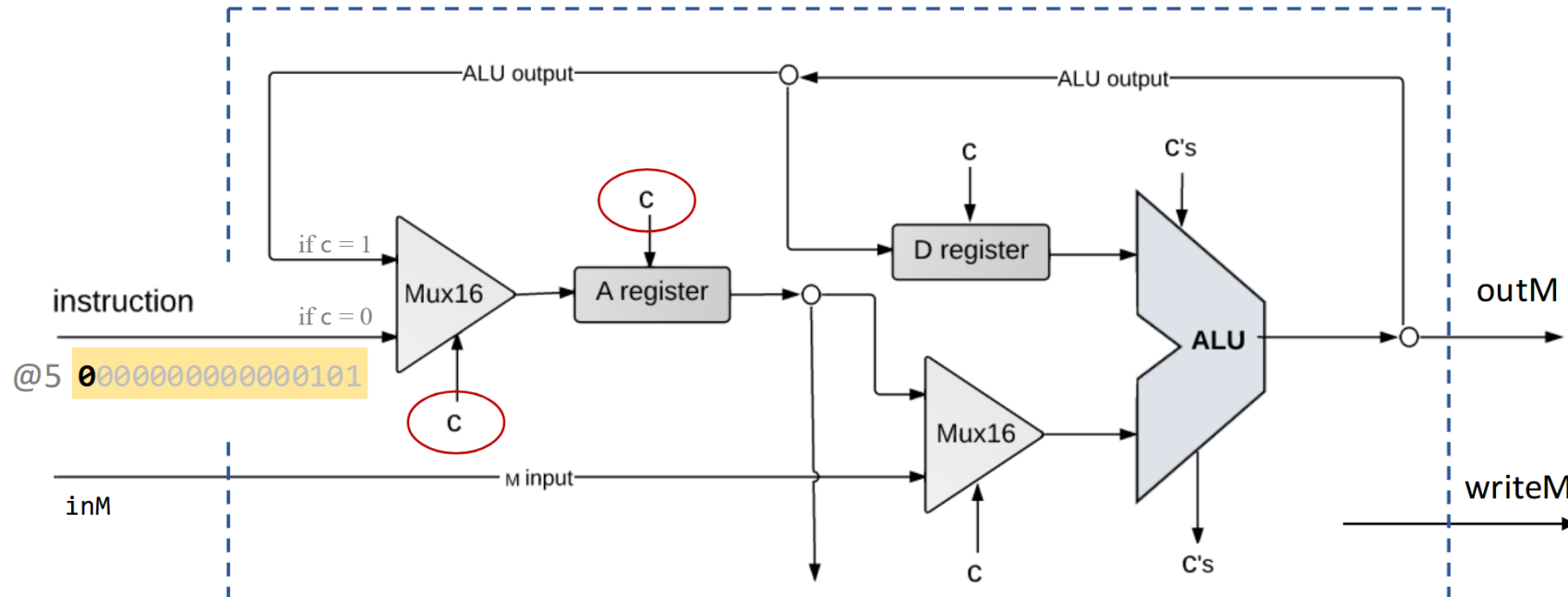
Hack CPU architecture

Example instructions:

@13
D=0
M=M+1
0;JMP

CPU: Instruction handling

@5



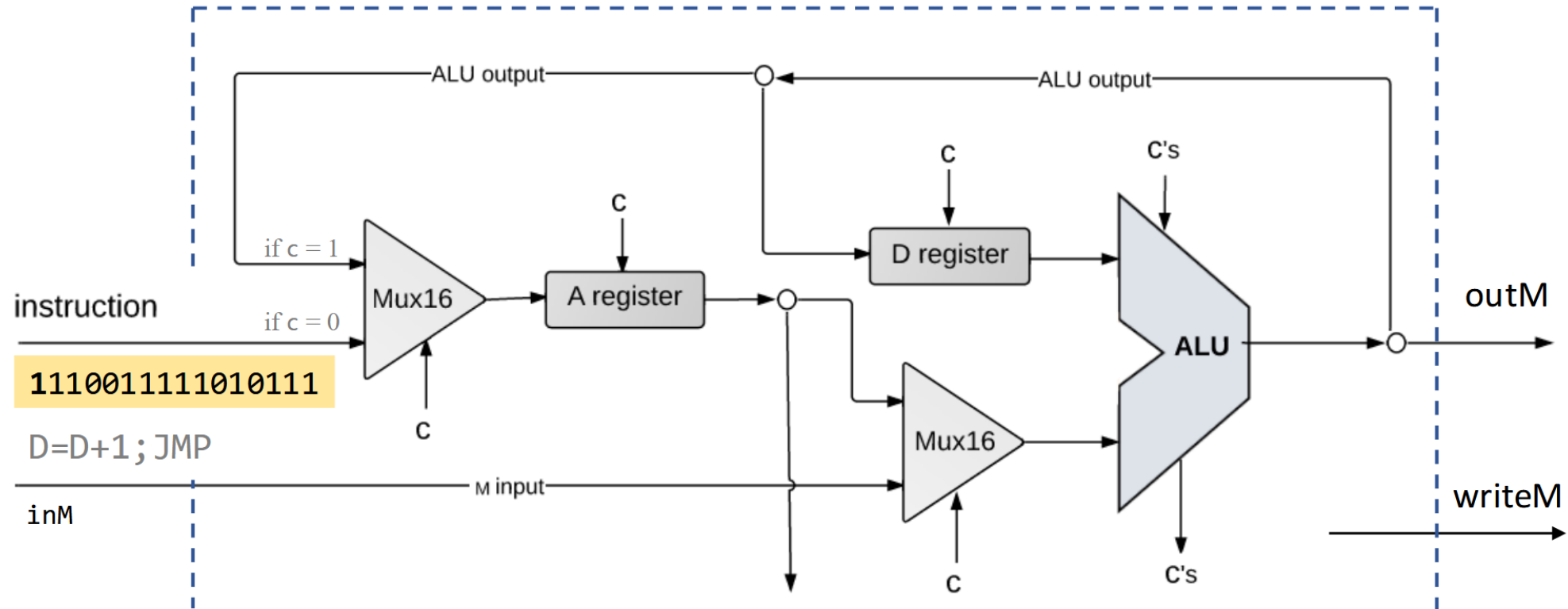
Handling A-instructions

Use the instruction's MSB (op-code) to manipulate the control bits of the A register and the Mux16 before it.

Effect: A-register $\leftarrow$ instruction (treated as a value)

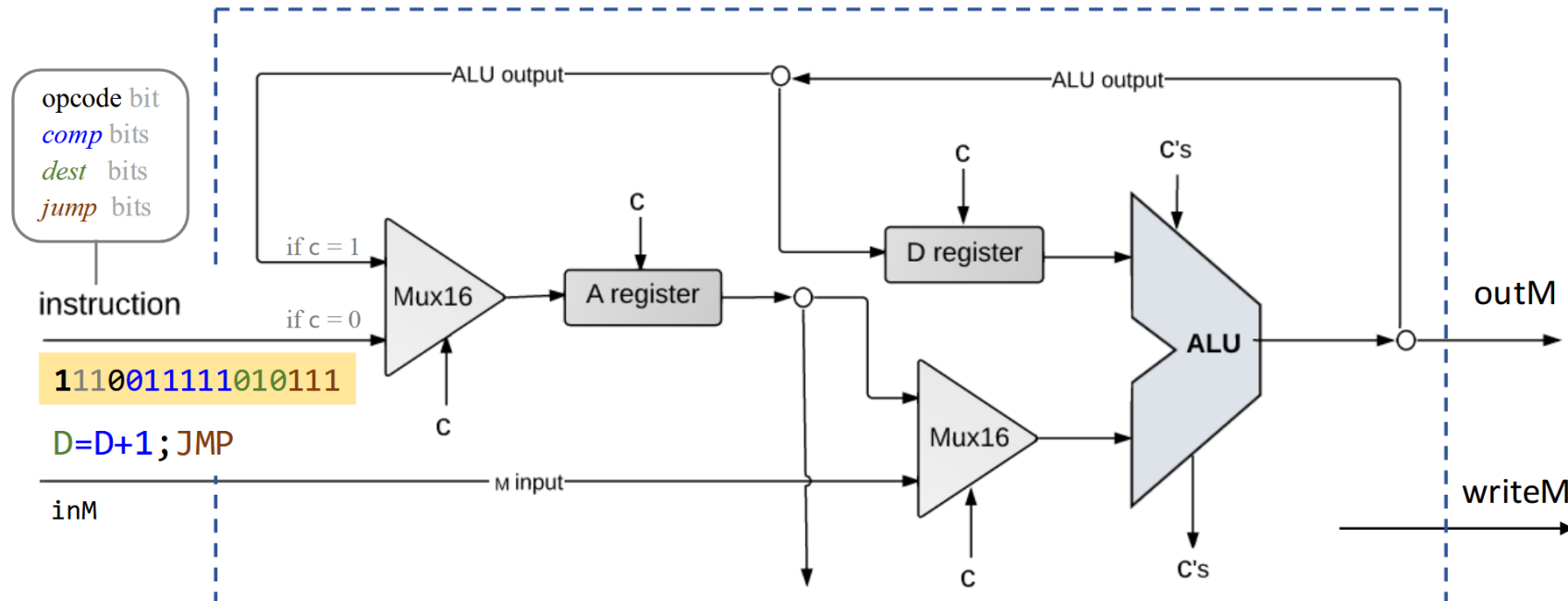
(Exactly what the @xxx instruction specifies: “set A to xxx”)

CPU: Instruction handling



Handling C-instructions

CPU: Instruction handling



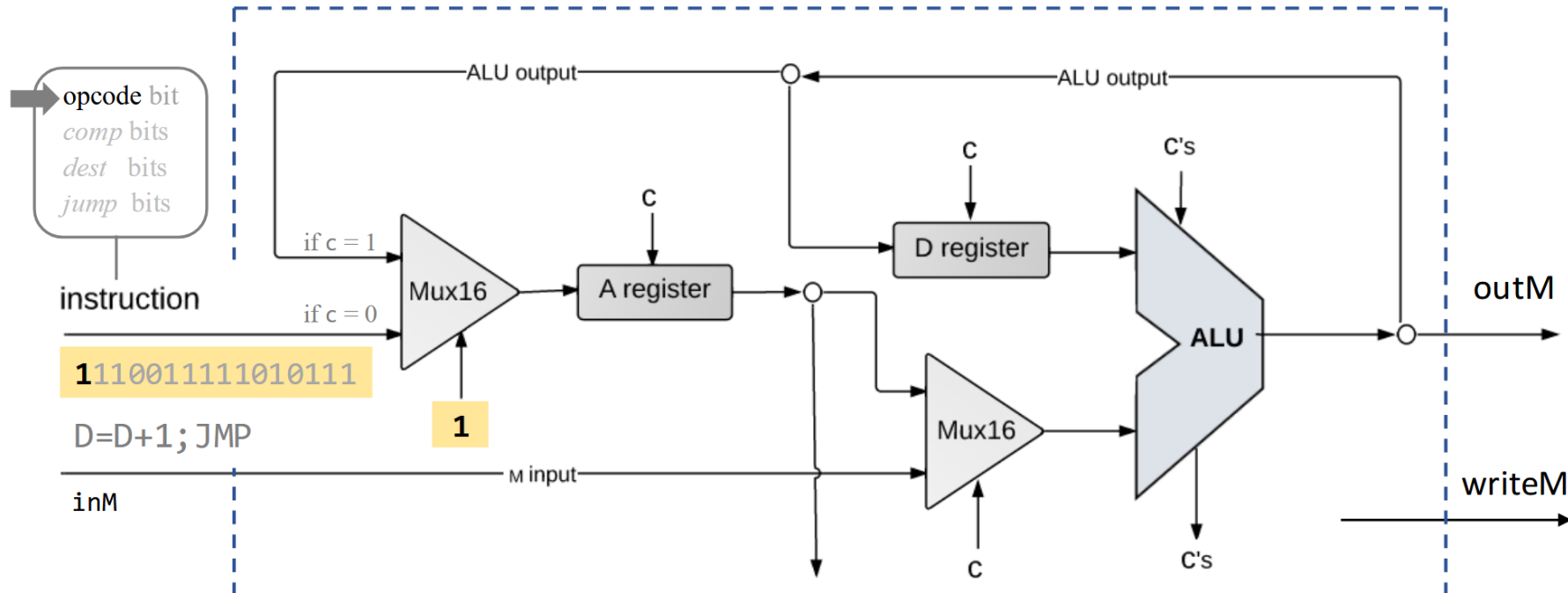
Handling C-instructions

Each instruction field (*opcode*, *comp*, *dest*, and *jump* bits) is handled separately

Each group of bits is used to "tell" a CPU chip-part what to do

Taken together, the chip-parts end up executing the instruction.

CPU: Instruction handling

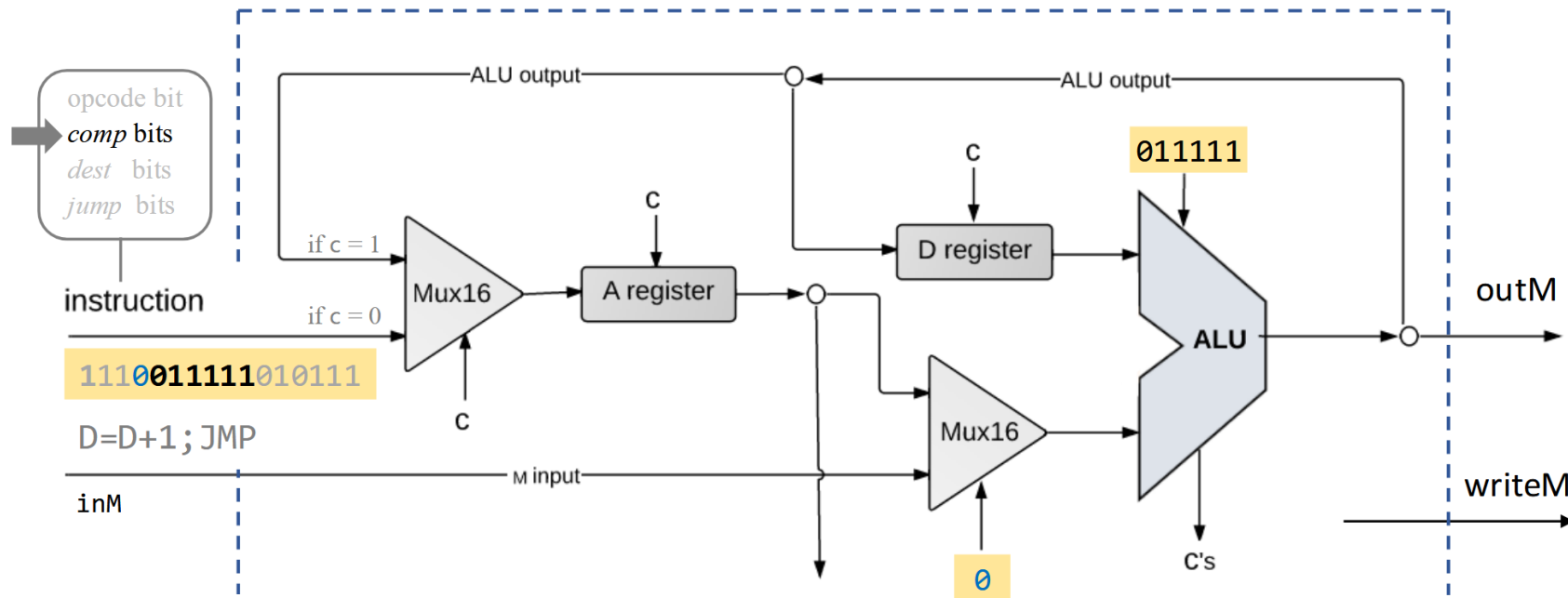


Handling C-instructions: The *opcode* bit

Routes the instruction's MSB to the Mux16

Effect: Primes the A register to get the ALU output.

CPU: Instruction handling

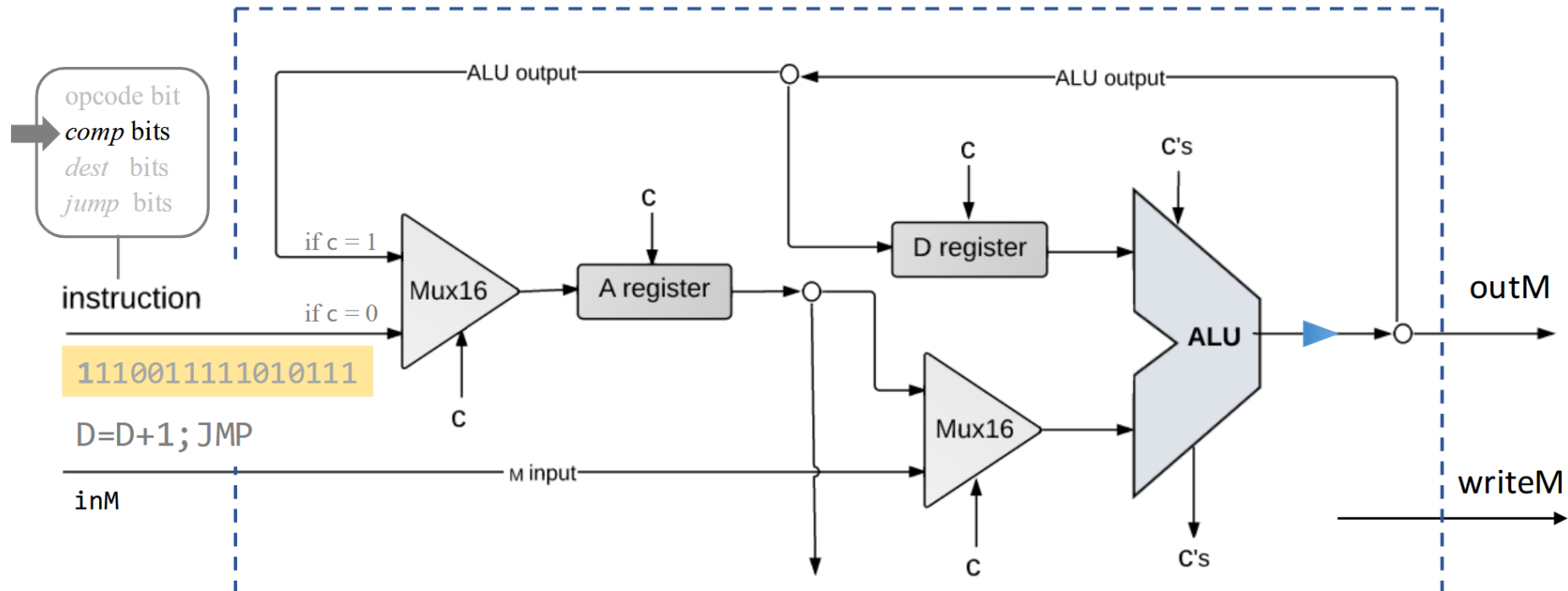


Handling C-instructions: The *computation* bits

- Routes the instruction's c-bits to the ALU control bits
- Routes the instruction's a-bit to the Mux16

Effect: the ALU computes the specified function
and emits the resulting value to the ALU output

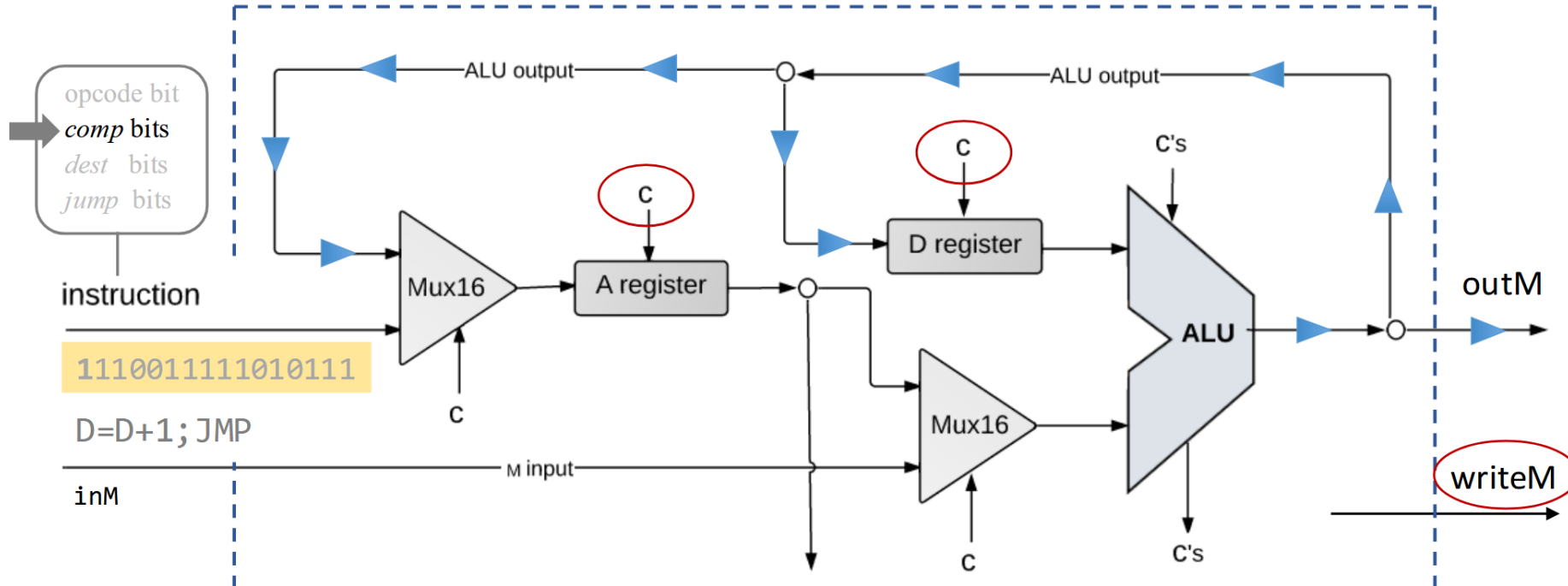
CPU: Instruction handling



ALU output:

- Result of ALU calculation

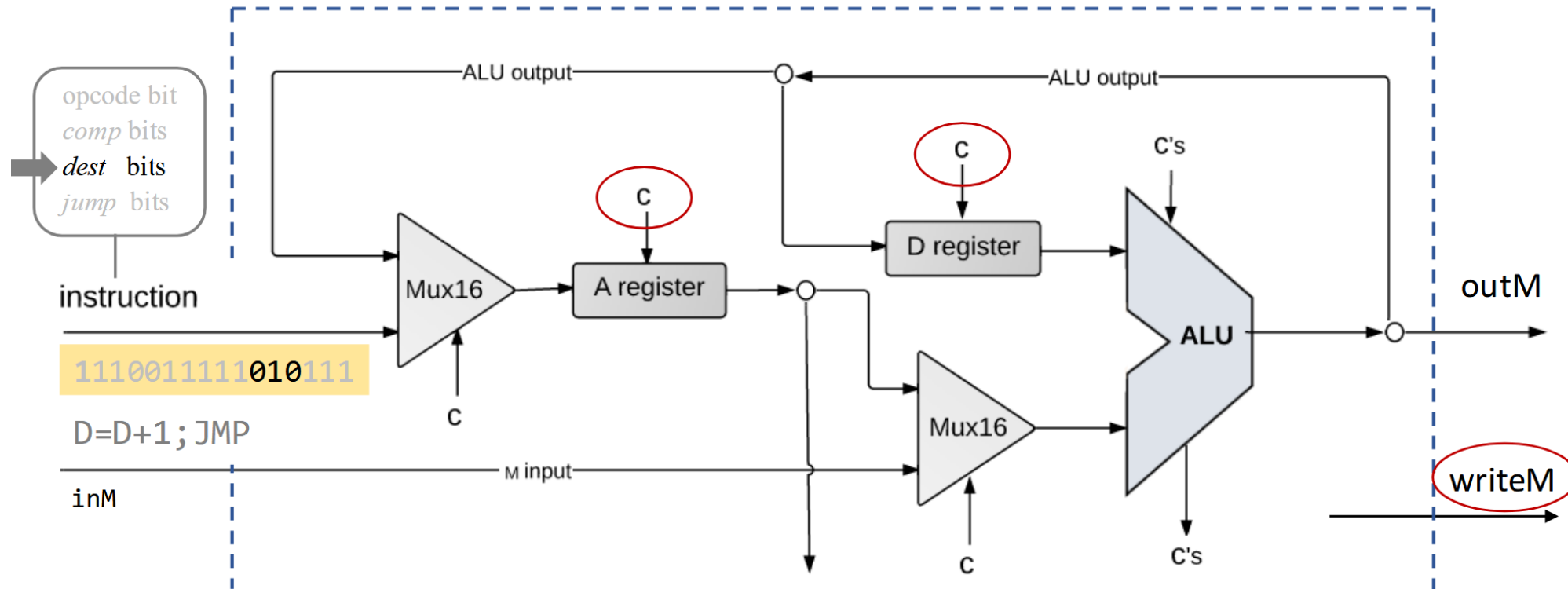
CPU: Instruction handling



ALU output:

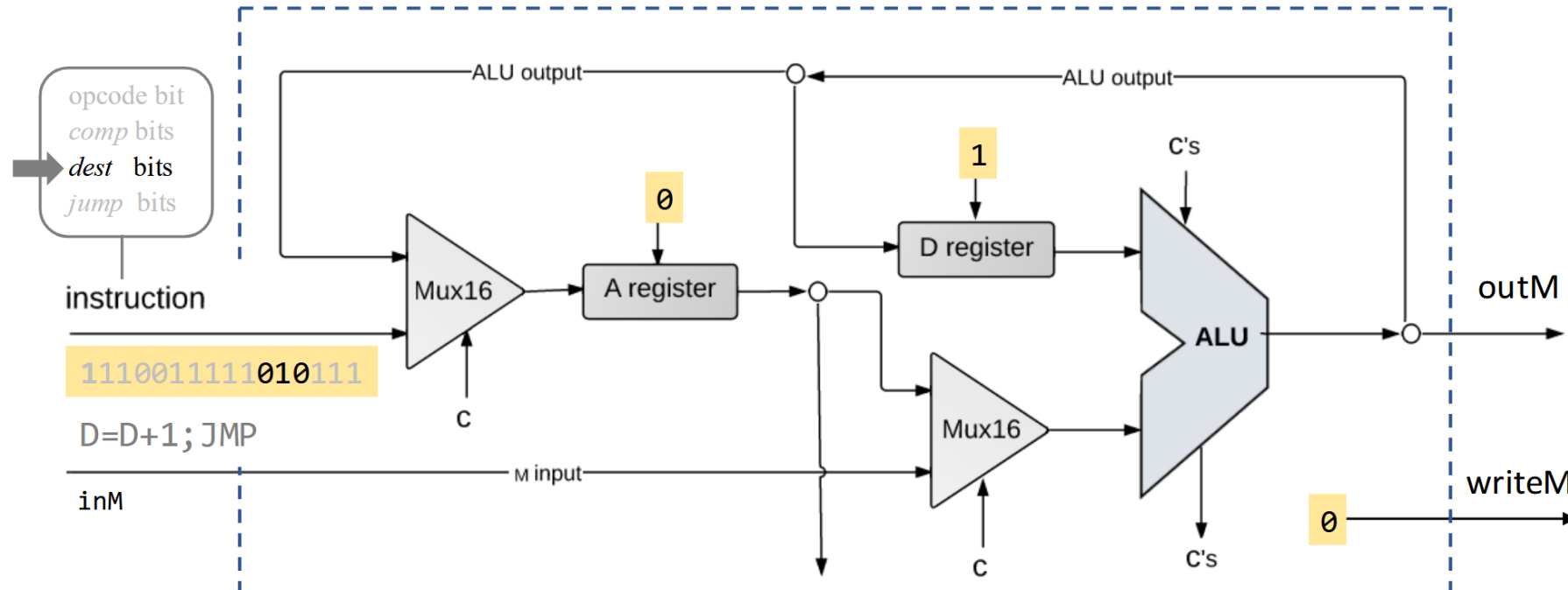
- Result of ALU calculation
- Fed simultaneously to D-register, A-register, data memory
- Each enabled/disabled by its control bit

CPU: Instruction handling



Handling C-instructions: The *destination* bits

CPU: Instruction handling

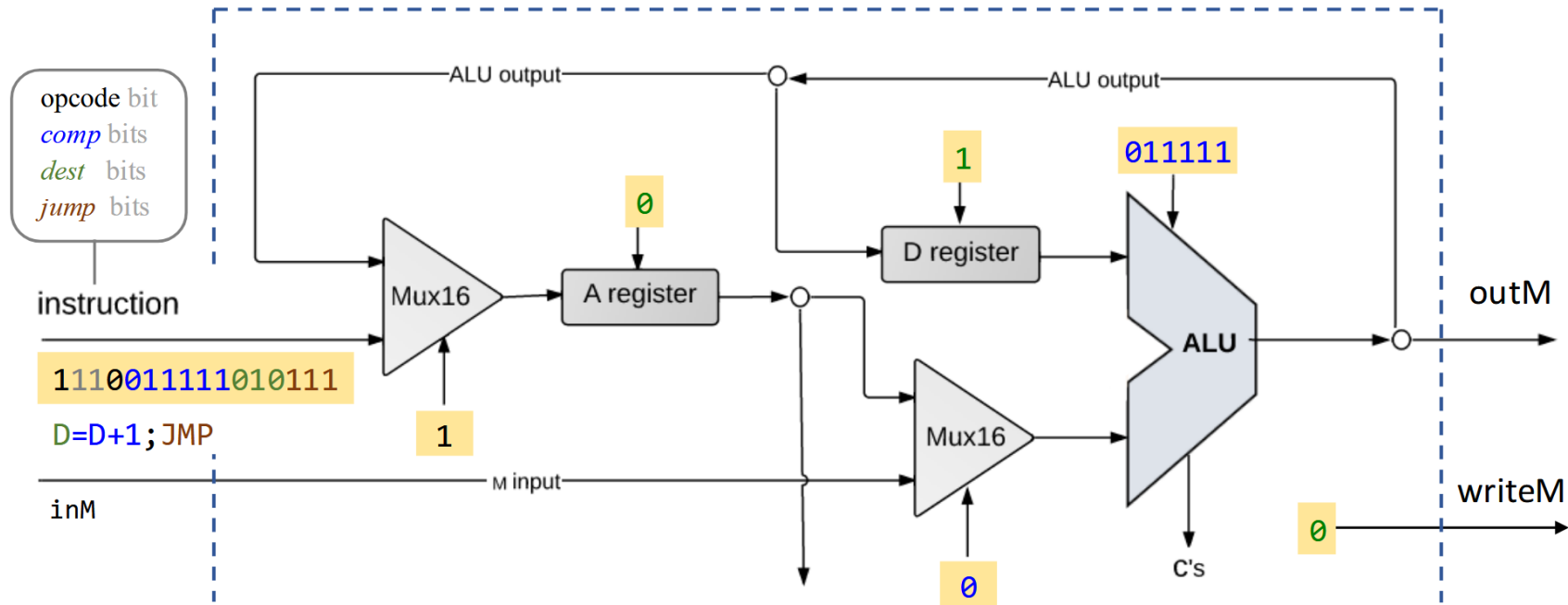


Handling C-instructions: The *destination* bits

Routes the instruction's d-bits to the control (load) bits of the A-register, D-register, and to the writeM bit

Effect: Only the enabled destinations commit to the ALU output

CPU: Instruction handling



Handling C-instructions: Recap

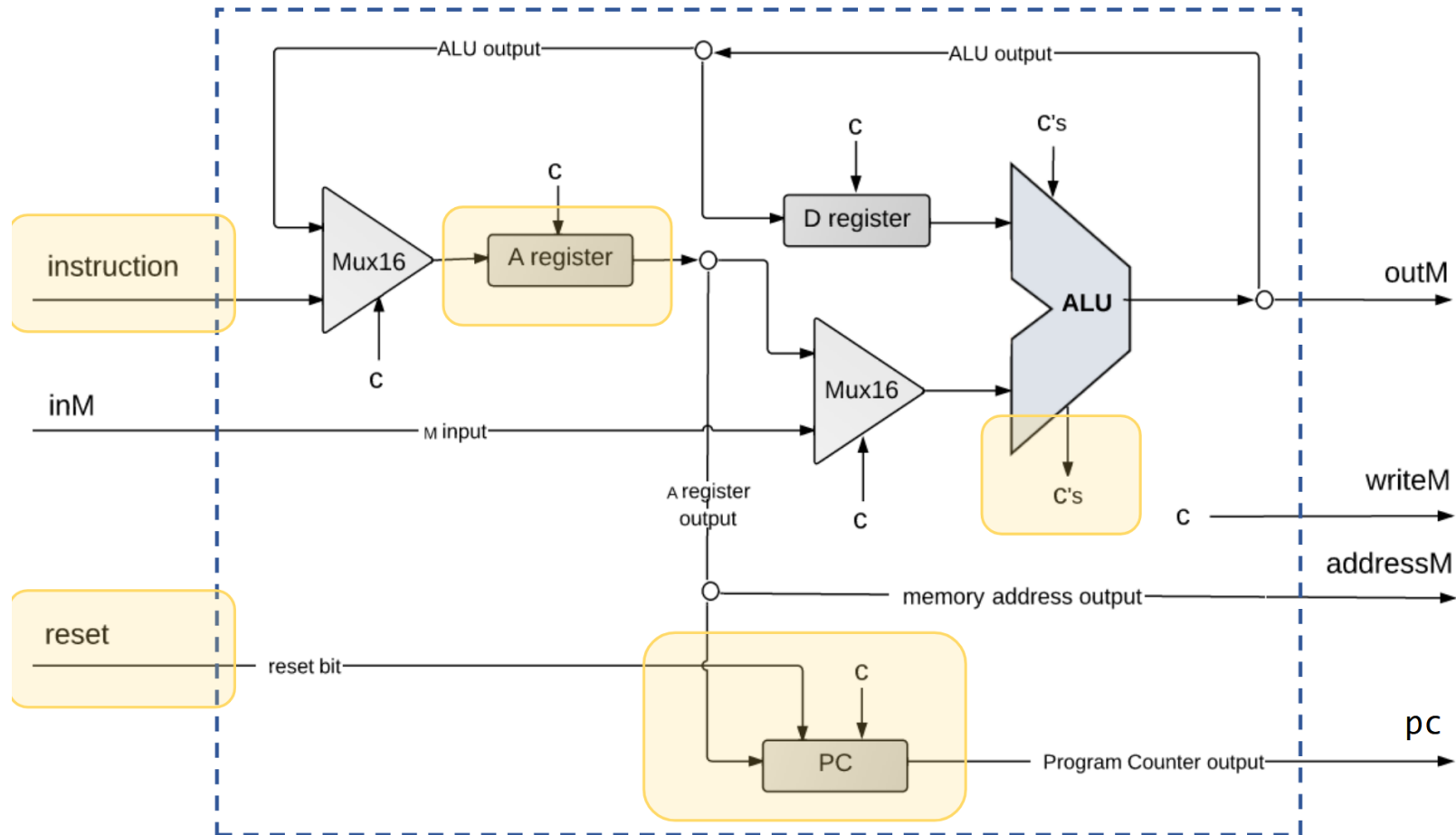


Executes $dest = comp$



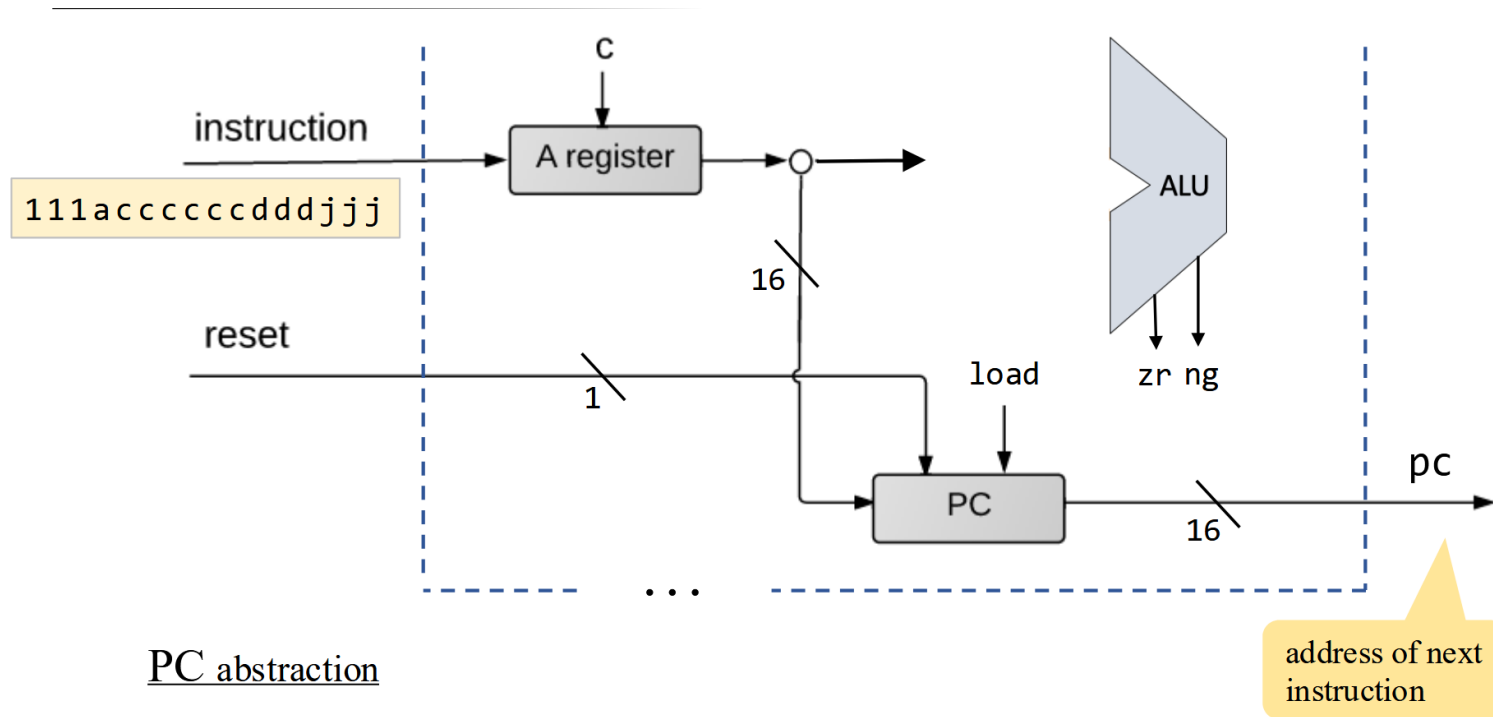
Figures out which instruction to execute next

CPU: Control



Hack CPU architecture

CPU: Control



PC abstraction

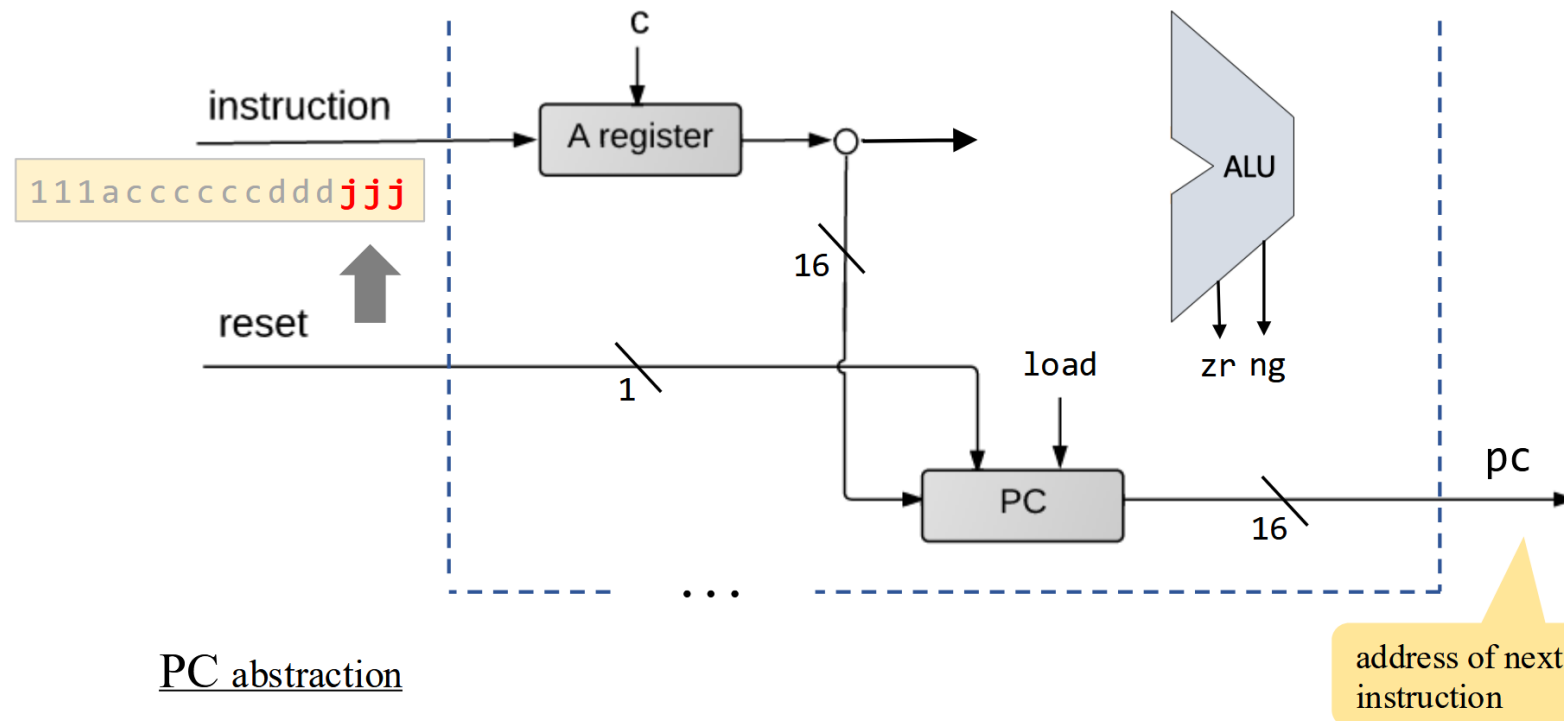
Outputs the address of the next instruction, has three states:

reset: $PC \leftarrow 0$

no jump: $PC++$

jump: $\text{if}(\text{condition}) PC \leftarrow A$ // Note: A was already set to the jump address

CPU: Control



PC abstraction

Outputs the address of the next instruction, has three states:

reset: $PC \leftarrow 0$

no jump: $PC++$

jump: if (*condition*) $PC \leftarrow A$ // Note: A was already set to the jump address

CPU: Control

Symbolic
syntax:

dest = comp ; jump

Binary
syntax:

1 1 1 a c c c c c c d d d j1 j2 j3

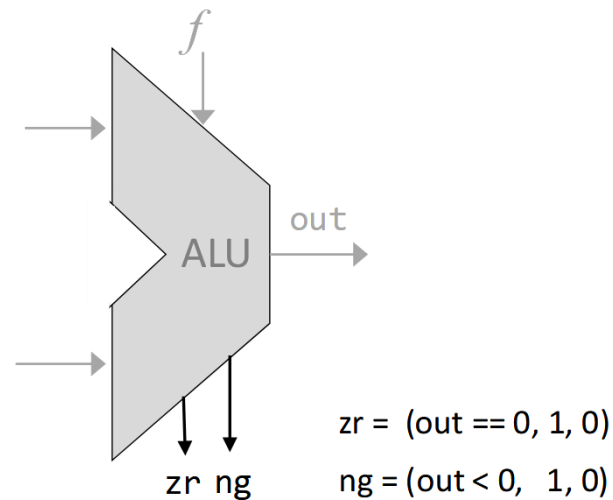
Compute expressions for
the following

a) J(0,0,1,zr, ng)

b) J(1,0,1,zr,ng)

c) J(1,1,1,zr,ng)

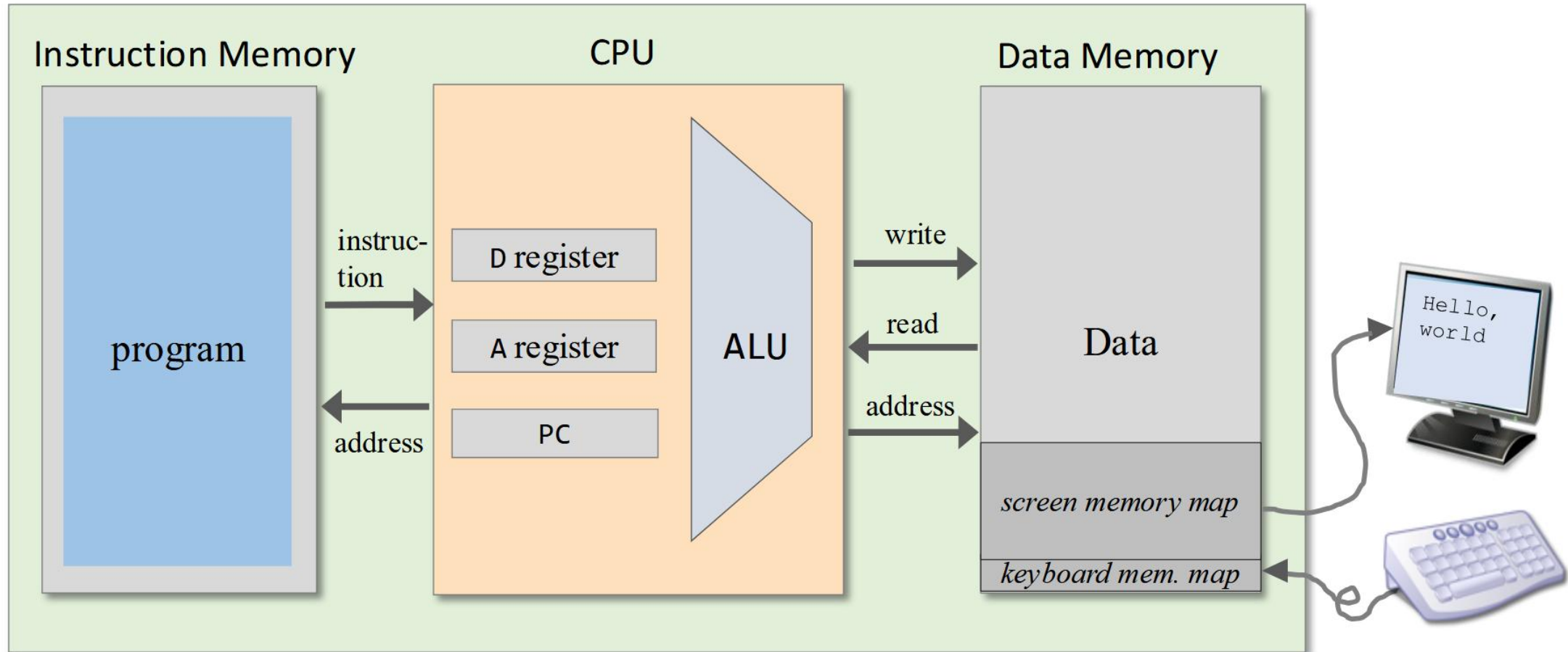
<i>jump</i>	<i>j1</i>	<i>j2</i>	<i>j3</i>	<i>condition</i>
null	0	0	0	no jump
JGT	0	0	1	if (ALU out > 0) jump
JEQ	0	1	0	if (ALU out = 0) jump
JGE	0	1	1	if (ALU out ≥ 0) jump
JLT	1	0	0	if (ALU out < 0) jump
JNE	1	0	1	if (ALU out ≠ 0) jump
JLE	1	1	0	if (ALU out ≤ 0) jump
JMP	1	1	1	Unconditional jump



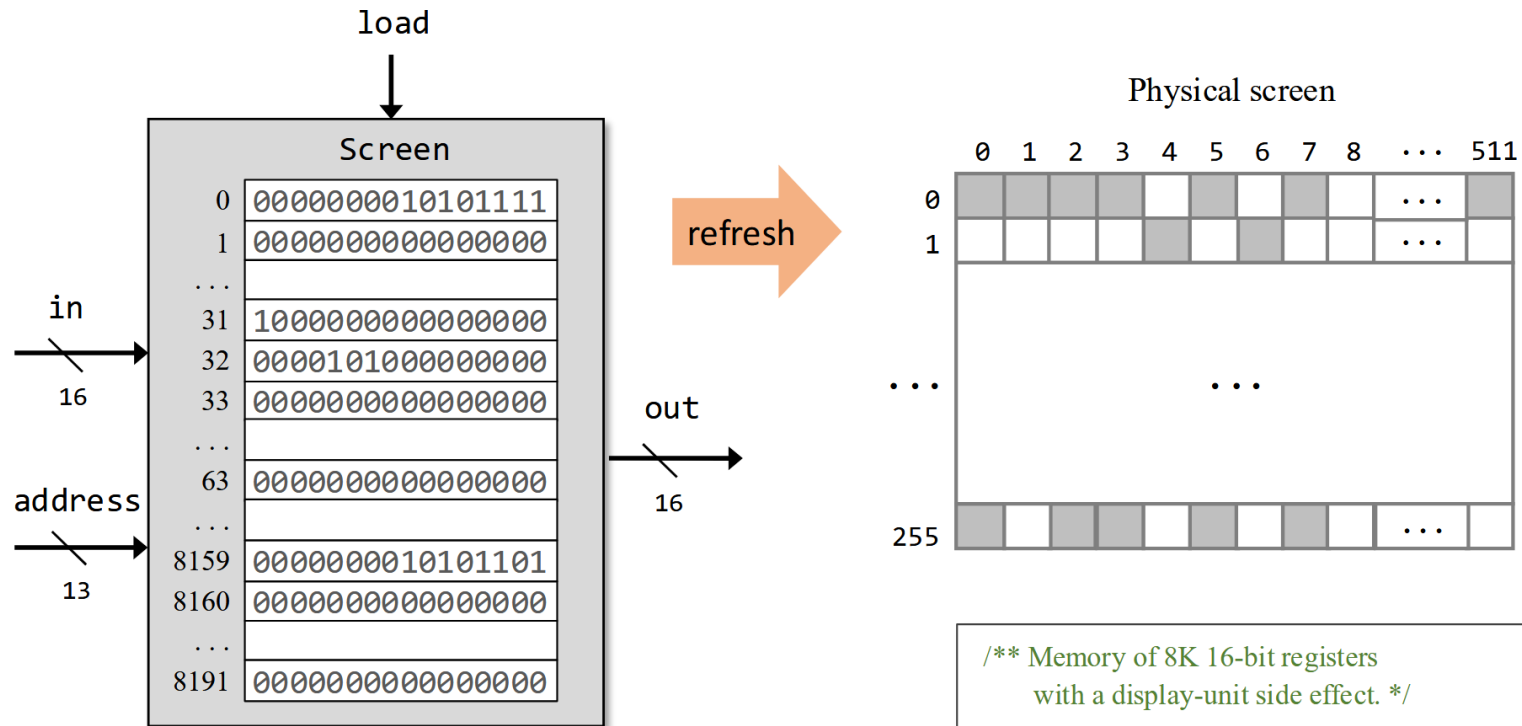
We can use gate logic to compute:

$J(j1, j2, j3, zr, ng) = 1$ if *condition* is true
0 otherwise

Computer implementation



Screen



Implemented as a built-in 8K memory chip named Screen

Memory-mapped I/O
associates values in memory with a device

Idea: Reading and writing to the device is done the same as reading and writing to memory

```
/** Memory of 8K 16-bit registers
    with a display-unit side effect. */
CHIP Screen {
    IN  address[13], in[16], load;
    OUT out[16];
    BUILTIN Screen;
    CLOCKED in, load;
}
```

Screen coordinates

The SCREEN starts at address 16384 (0x4000)

256 rows with 512 pixels each

1 → black

0 → white

The pixel location at row r and column c is given by the formula:

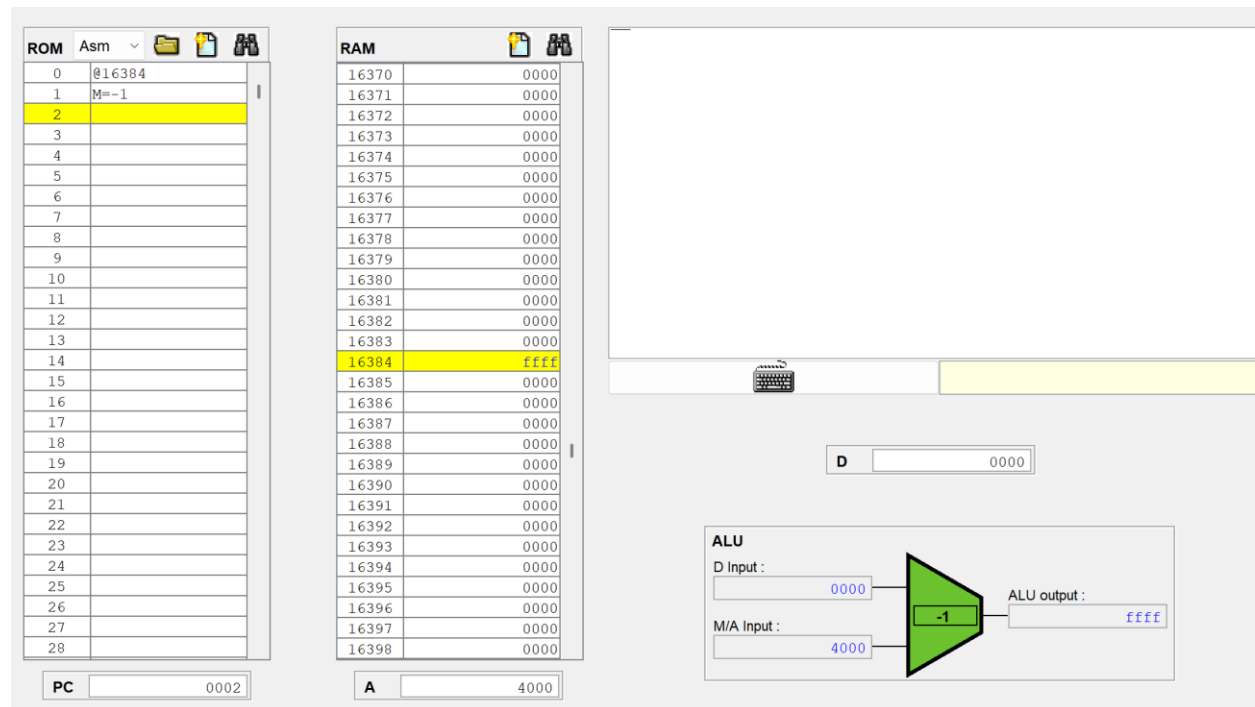
$\text{RAM}[\text{SCREEN} + r * 32 + c/16]$

Screen - Assembly

The SCREEN keyword marks the address of the beginning of screen memory

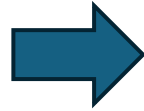
Example: The following program sets the first 8 pixels to black

```
@SCREEN  
M=-1
```



Screen – Setting the screen to black

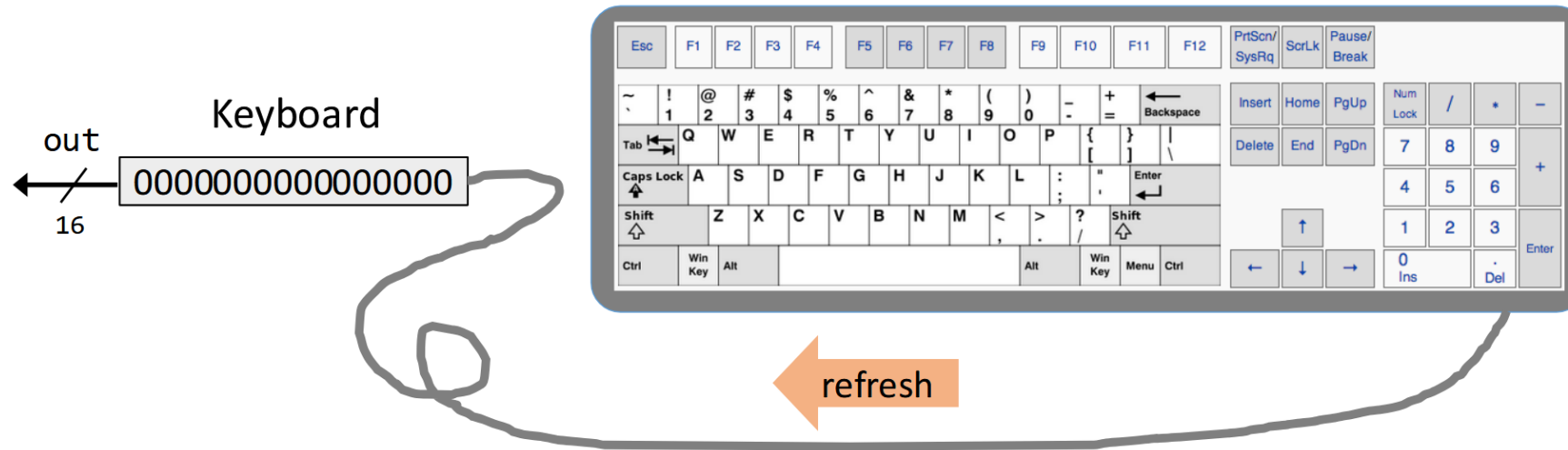
```
i = 0
LOOP:
if (i == 8192) goto END
M[SCREEN+i] = -1
i = i+1
END:
// infinite loop
```



```
// i = 0
@i
M=0
(LLOOP)
// if (i == 8192) goto END
@8192
D=A
@i
D=D-M
@END
D;JEQ
// M[SCREEN + i] = -1
@i
D=M
@SCREEN
A=D+A
M=-1
```

```
// i = i + 1
@i
M=M+1
@LOOP
0;JMP
(END)
@END
0;JMP
```

Keyboard



Implemented as a built-in
16-bit memory register
named Keyboard

```
/** 16-bit register, outputs the character code of the currently  
    pressed keyboard key, or 0 if no key is pressed */
```

```
CHIP Keyboard {  
    OUT  
        out[16];  
    BUILTIN Keyboard;  
}
```

Keyboard – Assembly Example

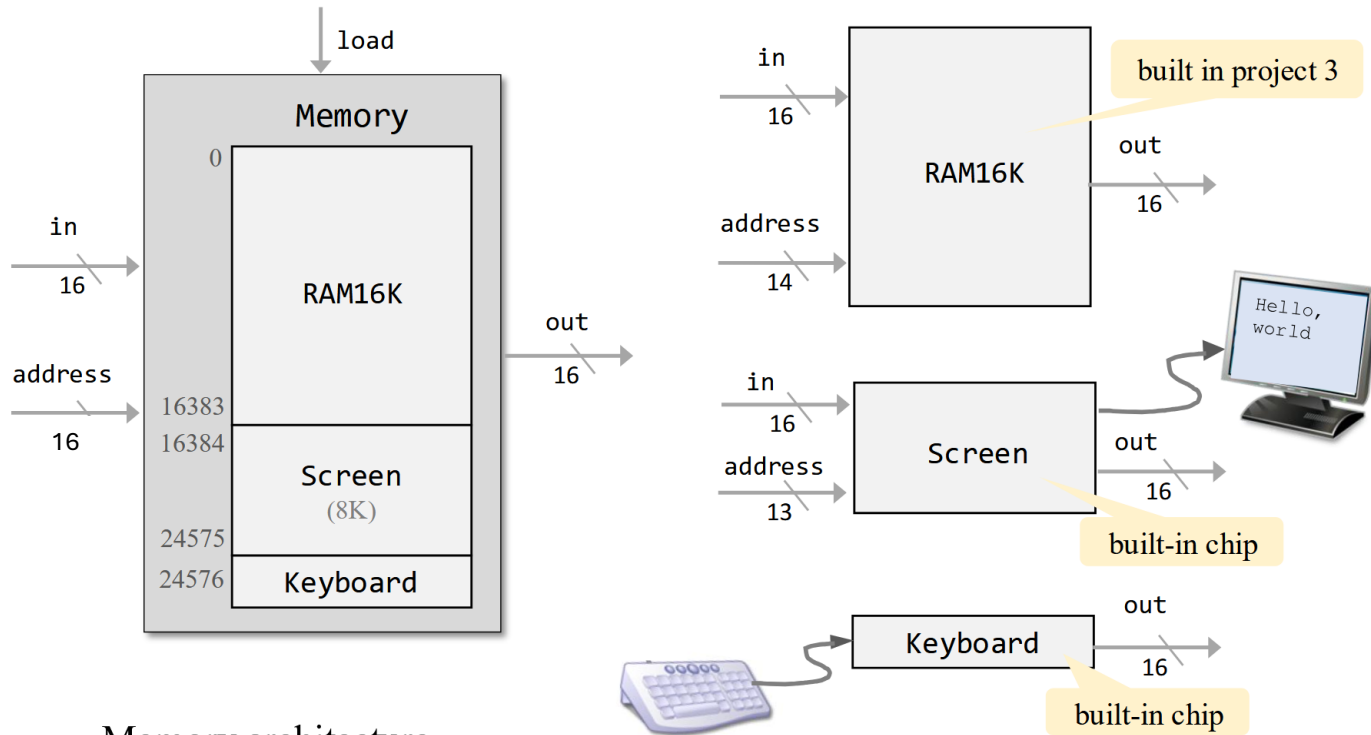
```
(LOOP)
    @KBD
    D=M
    @KEY_PRESSED
    D;JNE
    @LOOP
    0;JMP

(KEY_PRESSED)
    // set screen to black
```

If a key has been pressed since the last clock cycle, its ASCII code will be in M[KBD]

Otherwise, M[KBD] = 0

Memory



Memory architecture

- An aggregate of three chip-parts: RAM16K, Screen, Keyboard
- Single address space, 0 to 24576 (0x6000)
- Maps the address input onto the address input of the relevant chip-part.

Putting it altogether: Computer

